Dynamic Threshold Key Encapsulation with Transparent Setup

Joon Sik Kim[1], Kwangsu Lee*[2], Jong Hwan Park[3], and Hyoseung Kim*[4]

[1]Korea University
[2]Sejong University
[3]Sangmyung University
[4]Hallym University

Abstract

A threshold key encapsulation mechanism (TKEM) facilitates the secure distribution of session keys among multiple participants, allowing key recovery through a threshold number of shares. TKEM has gained significant attention, especially for decentralized systems, including blockchains. However, existing constructions often rely on trusted setups, which pose security risks such as a single point of failure and are limited by fixed participant numbers and thresholds. To overcome this issue, we propose a dynamic TKEM with a transparent setup, allowing for a flexible selection of both recipients and thresholds without relying on trusted third parties in the setup phase. In addition, our construction does not rely on pairing operations, which are less efficient compared to exponentiation. We prove the selective chosen-ciphertext security of our construction under the decisional Diffie-Hellman assumption, zero-knowledge, and soundness of a non-interactive zero-knowledge (NIZK) proof system. We also show that our scheme satisfies decapsulation consistency when the underlying NIZK system is sound. Our proof-of-concept implementation highlights the practicality and efficiency of this approach, further advancing the field of threshold cryptography.

1 Introduction

A threshold key encapsulation mechanism (TKEM) is a useful primitive in threshold cryptosystems, which distributes a session key among multiple parties. In the concept of TKEM with threshold parameters (n,t), the session key is split into n shares and encapsulated as a ciphertext header. Collecting at least t shares allows decapsulation of the header by reconstructing the original session key. TKEMs must fulfill two security requirements: chosen-ciphertext attack (CCA) security and decapsulation consistency (DC). Briefly, CCA security ensures that no adversary with fewer than t shares can gain any useful information about a session key corresponding to a given ciphertext header. A weaker notion, known as selective CCA security, applies only to a specific set of recipients chosen in advance. DC guarantees that any valid set of t shares out of n will consistently reconstruct the same session key.

Dynamic TKEMs allow the parameters (n,t) to be determined during encapsulation. This flexibility ensures that the number of participating parties and the required shares for decapsulation can change as needed. Furthermore, a transparent setup allows for the generation of public parameters without any secret

*The two authors are the corresponding authors of this paper.

Table 1: Comparison with the existing dynamic threshold cryptosystems

	Size			Dynamic	Transparent	Pairing	Method	Security	Assumption
	PP	*PK&SK*	*CT*	Threshold	Setup	Free		Model	
DP08 [19]	$O(N)$	$O(1)$	$O(1)$	△	X[†]	X	KEM	ROM	MSE-DDH
SES16a [42]	$O(1)$	$O(1)$	$O(n)$	O	O	X	PKE[§]	STM	DBDH/DLIN
SES16b [42]	$O(1)$	$O(1)$	$O(n)$	O	X	X	PKE	STM	DLIN
GKPW24 [28]	$O(N)$	$O(1)$	$O(1)$	△	X	X	PKE	ROM	GGM
Ours	$O(1)$	$O(1)$	$O(n)$	O	O	O	KEM	ROM	DDH

'O' indicates that the corresponding property is satisfied, while '△' indicates that it is satisfied with restrictions based on N (the maximum number of n participants). 'X' indicates that the corresponding property is not satisfied. ROM and STM refer to the random oracle model and the standard model, respectively. † requires key distribution with a higher level of trust. § indicates the conceptual difference where public verification of the well-formedness of CT is not allowed. Cryptographic assumptions in the last column are involved in the selective CCA security.

trapdoor, ensuring that no single party can compromise the system. This is crucial for mitigating the risk of a single point of failure, thus establishing TKEMs as essential in distributed systems such as blockchain and decentralized finance (DeFi) [14, 28].

Despite the importance of TKEM, only a few results on dynamic threshold cryptosystems have been proposed (e.g., [19, 28, 42]), focusing on threshold public key encryption (TPKE). TPKE is analogous to TKEM in that both require collaboration among multiple parties to reconstruct a message (in TPKE) or a key (in TKEM). Although they can be converted into each other, TKEM is advantageous for transmitting large messages when integrated with a data encapsulation mechanism [44].

1.1 Existing Dynamic TKEM & TPKE schemes

The first dynamic TKEM was constructed by Delerablée and Pointcheval [19], referred to here as DP08. While providing constant-size ciphertexts regardless of the number of participants, their construction relies on pairing operations. Moreover, DP08 has limitations in several respects: the dynamic threshold property is restricted, as the maximum number of n participants must be fixed during the initial setup stage. Additionally, a trusted setup is required, involving a master key to distribute n secret keys. Regarding security, their selective CCA security is proven based on the multi-sequence of exponent Diffie-Hellman (MSE-DDH) assumption, which is a q-type assumption. Unfortunately, a larger n requires a larger parameter q. This leads to stronger assumptions and ultimately limits scalability. Constructing TPKE is another research topic in threshold cryptography. Notably, two fully dynamic threshold TPKE schemes, denoted as SES16a and SES16b, were presented in [42]. Both are proven secure based on the standard assumption, such as the decision linear (DLIN) assumption and the decisional bilinear Diffie-Hellman (DBDH) assumption, in the standard model. This makes their constructions theoretically stronger than those proven in the random oracle model (ROM). However, both depend on pairing operations. In particular, SES16b presents a concrete construction following the design paradigm described earlier (by utilizing Groth-Sahai systems), thus requiring a trusted setup. On the other hand, SES16a provides a pairing-based framework that enables transparent setup. This originates from the required primitives, such as one-time signatures, commitment schemes, and PKENO, none of which need the help of a trusted setup. However, the syntax of SES16a differs slightly from others, even though TPKE is compatible with TKEM. The main difference is that the

well-formedness of ciphertexts (in the context of TPKE) could not be publicly verified. Recently, Garg et al. [28] proposed a remarkable TPKE scheme, denoted as GKPW24, achieving constant-size ciphertexts. In contrast to DP08, this particularly removes the need for distributed key generation. This advantage is largely due to the succinctness of the underlying Kate–Zaverucha–Goldberg polynomial commitment scheme [33]. However, such a commitment scheme relies on structured reference strings (SRS) that embed a trapdoor, thus requiring trust to ensure the trapdoor is not leaked. Indeed, in GKPW24, revealing the trapdoor can undermine even the semantic security of the scheme. Moreover, its security is proven in the generic group model, which is viewed as less robust than the standard model and ROM.

To the best of our knowledge, no existing scheme simultaneously supports dynamic threshold, transparent setup, and pairing-free operations, as shown in Table 1.

1.2 Our results

We propose a dynamic TKEM with a transparent setup, which is constructed without the need for pairing operations. Moreover, the proposed TKEM meets the security requirements, which are proven based on the decisional Diffie-Hellman assumption and underlying cryptographic primitives, in the random oracle model. We remark that this is the first result on threshold cryptosystems. To do this, we focus on the following steps:

Dynamic Threshold: To achieve this, we employ the approach originating from [42], which utilizes a public key encryption (PKE) scheme, non-interactive zero-knowledge (NIZK) proof systems, and a polynomial-based secret sharing scheme. The secret sharing scheme is generally used to support the (n,t)-threshold setting. More precisely, a polynomial f of degree $t-1$ is used to share the secret $s_0 = f(0)$ by splitting it into n shares as $f(idx_1),\ldots,f(idx_n)$, where idx_i is assigned to a user i. Any t shares out of n can be used to reconstruct the secret by interpolation: $s_0 = \sum L_i(0) f(idx_i)$, where $L_i(0)$ is the Lagrange coefficient. To be dynamic, such a polynomial and its shares are now generated during encapsulation. Each share is encrypted using the PKE scheme under its corresponding public key and then included in the ciphertext header. The resulting ciphertext header additionally contains an NIZK proof, ensuring that the ciphertext header is well-formed; that is, the corresponding secret is correctly shared, and the shares are properly encrypted. Note that the construction in [42], following the above approach, employs a pairing-based PKE with a non-interactive opening (PKENO) scheme and a Groth-Sahai proof system [29], which relies on a trusted setup that we aim to avoid.

Transparent Setup: As we explained before, Groth-Sahai proof systems are problematic because they require a trusted setup to guarantee that public structured reference strings are generated without a trapdoor for real-world use. Since the trusted setup demands significant computational effort, and such strings can be subverted as shown in [8], we address this problem by eliminating Groth-Sahai proof systems throughout our construction. One could consider applying NIZK proof systems derived from Sigma protocols with the Fiat-Shamir transformation [25], where the generation of strings is publicly verifiable, yielding a transparent setup. This is, however, not easily achieved, as Groth-Sahai proof systems are designed to prove complex mathematical structures. For instance, consider the form $M \cdot g^{f(idx_1)}$, where $M \in \mathbb{G}$ is a group element and $f(idx_1) \in \mathbb{Z}_p$ is a scalar. Fiat-Shamir NIZK proof systems from Sigma protocols are limited to proving knowledge of scalars; consequently, they are incapable of proving joint knowledge over heterogeneous algebraic structures such as $(M, f(idx_1))$. Hence, we design our TKEM so that only scalars need to be proven while ensuring the well-formedness of our ciphertext header. To this end, we focus on the work of [21], originally used for constructing signature-based witness encryption. This literature demonstrates the way to prove that polynomial-based shares as exponents over a group $\mathbb{G}$, as demonstrated above, are correct to reconstruct the secret via Fiat-Shamir NIZK proof systems from Sigma protocols. However, in [21],

pairing operations are still needed for proving other aspects of well-formedness, such as the structure of the BLS scheme [10]. We carefully extend this proof system to be an entirely pairing-free construction, as described below.

Pairing-Free Construction: Pairing operations are relatively more expensive than exponentiations over a conventional group. To eliminate the dependency on pairings, we employ a technique for double ElGamal encryption with shared randomness, which is a well-known solution for constructing a CCA-secure PKE scheme. This decision is mainly due to the fact that the structure of such ElGamal ciphertexts can be efficiently proven by statements for the equality of discrete logarithms. These statements can be ensured by using Fiat-Shamir NIZK proof systems from Sigma protocols that we aim to utilize. Furthermore, the security of the employed PKE scheme contributes to the selective CCA security of TKEM, as shown in [42]. Based on this, our ciphertext header contains variant ElGamal ciphertexts to encrypt shares of participants. Finally, we elaborately incorporate statements to prove its well-formedness (i.e., that the encrypted shares are all correct). A detailed description will be demonstrated in Section 3.2. In implementation, this pairing-free design achieves significantly lower cycle costs compared with SES16a, a pairing-based scheme that supports both dynamic thresholds and a transparent setup. As shown in Section 5.2, our TKEM reduces the encapsulation cost by a factor of more than 15.

In fact, our approach results in a TKEM producing a ciphertext header of size $O(n)$, where n is the number of participants, leading to computational inefficiency. This comes from the fact that neither ElGamal ciphertexts nor NIZK proofs can be aggregated. Nevertheless, the proposed TKEM could be a step towards future constructions with constant-size ciphertext headers. Moreover, as previously described, no existing construction offers the features outlined above.

1.3 Related Works

A threshold cryptography was first introduced in [20]. Since then, beyond TPKE and TKEM, it has also been extensively studied in the context of threshold signatures (TS), such as [6,7,9,17,36,41,46]. These schemes are proven secure under the discrete logarithm problem in the ROM. However, they require a trusted setup, either by executing a distributed key generation protocol or by assigning a designated party to generate keys for all participants. Conversely, certain pairing-based TS schemes [18, 27] enable a silent setup setting, where key generation is performed locally and the setup is completed without interaction. Nevertheless, as noted in [28], a minimal level of trust remains necessary to generate an SRS. Another line of research has investigated lattice-based TS schemes for post-quantum security (e.g., [2,13,22,23,30,39]). However, none of these constructions achieve a transparent setup.

2 Preliminaries

In this section, we provide the necessary prior knowledge to understand our paper.

2.1 Notations

For $n \in \mathbb{N}$, $[n]$ denotes the set $\{1,\ldots,n\}$. For $a,b \in \mathbb{Z}$ such that $a < b$, $[a,b]$ denotes the set $\{a,\ldots,b\}$. We especially use $\mathbb{Z}_p$ to denote a finite field with p elements, where p is a prime. A small bold letter means a column vector, whereas a large bold letter means a matrix. Given a vector $\mathbf{v}$, $\mathbf{v}^\top$ means the transposed $\mathbf{v}$. The meaning of $a \xleftarrow{\$} \mathbb{S}$ is the uniformly random sampling of the element a from the set $\mathbb{S}$.

2.2 Lagrange Interpolation

Lagrange interpolation is a way to recover shared secret values efficiently. Let $f(\gamma) = \sum_{j=0}^{t-1} s_j \gamma^j$ be a $(t-1)$-degree polynomial and let $(\gamma_1, f(\gamma_1)), \ldots, (\gamma_n, f(\gamma_n))$ be n points evaluated by the polynomial with $n > t$. Therein, all coefficients and evaluation points are assumed to be elements of $\mathbb{Z}_p$. Given any t points out of the n points, we can recover the secret value $s_0 = f(0)$: Specifically, without loss of generality, these t points are represented as $(\gamma_1, f(\gamma_1)), \ldots, (\gamma_t, f(\gamma_t))$. Then we can compute $s_0 = \sum_{i\in[t]} f(\gamma_i) L_i(0)$ where $L_i(0) = \prod_{j\in[t], j\neq i} \frac{-\gamma_j}{\gamma_i - \gamma_j}$ is the Lagrange basis at the point 0.

2.3 Reed-Solomon Codes

Reed-Solomon codes are denoted as $RS_{n+1,t}[\gamma]$ for a vector $\gamma = (\gamma_0, \ldots, \gamma_n)^\top \in \mathbb{Z}_p^{n+1}$, where $t \in [n]$. For each $(t-1)$-degree polynomial $f(\gamma) = \prod_{j=0}^{t-1} s_j \gamma^j$, we let $\mathbf{s} = (s_0 \ldots, s_{t-1})^\top \in \mathbb{Z}_p^t$ and $\mathbf{f} = (f(\gamma_0), \ldots, f(\gamma_n))^\top$. Then $RS_{n+1,t}[\gamma]$ consists of two matrices $\mathbf{G} = (\gamma_i^j)_{i,j} \in \mathbb{Z}_p^{(n+1)\times t}$ and $\mathbf{H} = (\frac{1}{\prod_{\ell\in[0,n],\ell\neq j} \gamma_j - \gamma_\ell} \gamma_j^i)_{i,j} \in \mathbb{Z}_p^{(n-t+1)\times(n+1)}$. The former is a generator matrix since $\mathbf{G}\cdot\mathbf{s} = \mathbf{f}$, and the latter is a parity-check matrix since $\mathbf{H}\cdot\mathbf{G} = \mathbf{0}$. We adopt such Reed-Solomon codes to verify that $\mathbf{f}$ was correctly generated by $\mathbf{G}$, as in [21]. Note that $\mathbf{f}$ is correctly generated only if the equation $\mathbf{H}\cdot\mathbf{f} = \mathbf{H}\cdot\mathbf{G}\cdot\mathbf{s} = \mathbf{0}$ holds.

2.4 Non-Interactive Zero-Knowledge Proofs

Non-interactive zero-knowledge (NIZK) proofs are defined with an NP-language $\mathcal{L}_\mathcal{R} = \{s \in \{0,1\}^* | \exists w : (s,w) \in \mathcal{R}\}$, where $\mathcal{R} \subseteq \{0,1\}^* \times \{0,1\}^*$ is an efficiently recognizable relation. If $s \in \mathcal{L}_\mathcal{R}$, then we call such an s true statement for a witness w; otherwise, we call it a false statement.

The NIZK proof system includes the following algorithms:

- **Prove**(CRS, s, w): It takes a common reference string CRS, a statement s, and a witness w as input, and outputs a proof π.
- **Verify**(CRS, s, π): It takes a common reference string CRS, a statement s, and a proof π as input, and outputs 1 when s is true and 0 otherwise.

In particular, we mainly focus on Fiat-Shamir NIZK proof systems from Sigma protocols. These systems need a hash function H, modeled as a random oracle. In this model, those two algorithms are denoted as $\mathbf{Prove}^H$ and $\mathbf{Verify}^H$, respectively. Moreover, as shown in [24], the properties of such an NIZK proof system are demonstrated as follows: **Completeness**: If $s \in \mathcal{L}_\mathcal{R}$, any proof π generated by the proving algorithm is accepted by the verification algorithm. We say that an NIZK proof system is considered complete if the following holds:

$$\Pr[(s,w) \in \mathcal{R}; \pi \leftarrow \mathbf{Prove}^H(CRS, s, w) : \mathbf{Verify}^H(CRS, s, \pi) = 1] = 1.$$

Zero-knowledge: There is a simulator S, not knowing a witness, which can generate a simulated proof that is indistinguishable from a real one. In an explicitly programmable ROM, the simulator $\mathcal{S}$ maintains a common state st that is updated with every query. Then, $\mathcal{S}$ operates in two distinct modes: (1) When invoked as $\mathcal{S}(1, st, q)$, it sets the random oracle output $h = H(q)$ and returns (h, st). (2) When invoked as $\mathcal{S}(2, st, s)$, it simulates a proof π for s and then returns (π, st). We now define the oracle pair $(\mathcal{S}_1, \mathcal{S}_2)$, where $\mathcal{S}_1(q)$ returns h from $\mathcal{S}(1, st, q)$, and $\mathcal{S}_2(s)$ returns π from $\mathcal{S}(2, st, s)$ if $(s,w) \in \mathcal{R}$. We say that an NIZK

proof system is zero-knowledge in the ROM if, for any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$, the advantage of $\mathcal{A}$ is negligible in λ as follows:

$$\mathbf{Adv}_{NIZK}^{ZK}(\lambda) = |\Pr[\mathcal{A}^{H,\mathbf{Prove}^H}(\lambda) = 1] - \Pr[\mathcal{A}^{S_1,S_2}(\lambda) = 1]|,$$

where both $\mathbf{Prove}^H$ and S are output $\perp$ if $(s,w) \notin \mathcal{R}$.

(Simulation) Soundness: For a false statement s, a cheating prover cannot falsely convince the honest verifier that s is true. We say that an NIZK proof system is sound in the ROM if, for any PPT adversary $\mathcal{A}$, the following advantage of $\mathcal{A}$ is negligible in λ:

$$\mathbf{Adv}_{NIZK}^{Sound}(\lambda) = \Pr[(s^*,\pi^*) \leftarrow \mathcal{A}^{S_1,\mathbf{Prove}^H}(\lambda) : s^* \notin \mathcal{L}_{\mathcal{R}} \wedge \mathbf{Verify}^{S_1}(CRS, s^*, \pi^*) = 1].$$

In this notion, the cheating prover may obtain simulated proofs for true statements, but must still be unable to produce a valid proof for any false statement. This makes simulation soundness a strictly stronger notion than ordinary soundness. Formally, we say that an NIZK proof system is simulation sound in the ROM if, for any PPT adversary $\mathcal{A}$, its advantage is negligible in λ as follows:

$$\mathbf{Adv}_{NIZK}^{S-Sound}(\lambda) = \Pr[(s^*,\pi^*) \leftarrow \mathcal{A}^{S_1,S_2}(\lambda) : (s^*,\pi^*) \notin L \wedge s^* \notin \mathcal{L}_{\mathcal{R}} \wedge \mathbf{Verify}^{S_1}(CRS, s^*, \pi^*) = 1],$$

where L is the list of pairs (s,π) corresponding to the sequence of queries to $\mathcal{S}_2$.

A proof system is said to be a *proof of knowledge* if the existence of a valid proof implies that the prover possesses a witness for the statement being proven. Formally, this is captured by the existence of an efficient extractor which, given oracle access to a successful prover, can recover a valid witness with non-negligible probability. In the case of Fiat-Shamir NIZK proof systems from Sigma protocols, the extractor relies on the rewinding technique, which is efficient for extracting a single witness. However, when t multiple witnesses must be extracted sequentially from the same adversary (acting as a prover), each extraction depends on prior random oracle responses. As a result, the standard rewinding approach may require exponential time, up to 2^t, rendering it impractical for large values of t [45]. To overcome this limitation, we adopt a straight-line extraction technique (e.g., Fischlin's online extractor [26]), which enables efficient extraction of multiple witnesses without rewinding. This leads to the following formalization.

Extractability: We say that an NIZK proof system is extractable with extraction error $\varepsilon_{\mathcal{E}}$ in the ROM if, for any PPT adversary $\mathcal{A}$, there exists a PPT algorithm $\mathcal{E}$ (called the straight-line extractor) such that the following holds with negligible probability $\varepsilon_{\mathcal{E}}$:

$$\begin{aligned}\Pr[(s^*,\pi^*) \leftarrow A^{S_1,\mathbf{Prove}^H}(\lambda); w^* \leftarrow \mathcal{E}(s^*,\pi^*,L_H) : (s^*,\pi^*) \notin L \\ \wedge (s^*,w^*) \in \mathcal{R} \wedge \mathbf{Verify}^{S_1}(CRS, s^*, \pi^*) = 1] = 1 - \varepsilon_{\mathcal{E}},\end{aligned}$$

where L_H and L denote the lists of pairs $(q,H(q))$ and (x,π), corresponding to the sequences of queries to S_1 to S_2, respectively.

2.5 Underlying Assumption and Lemma

Our security proof relies on the DDH assumption, a cryptographic complexity assumption, which is described as follows:

Assumption 1. (Decisional Diffie-Hellman assumption, DDH). Let $\mathbb{G}$ be a cyclic group of prime order p and g be a random generator of $\mathbb{G}$. We say that the DDH assumption holds, if for any PPT adversary $\mathcal{A}$, the following advantage is negligible in λ:

$$\begin{aligned}\mathbf{Adv}_{DDH}(\lambda) = |\Pr[g \xleftarrow{\$} \mathbb{G}, \alpha, \beta \xleftarrow{\$} \mathbb{Z}_p : \mathcal{A}(\mathbb{G}, p, g, g^{\alpha}, g^{\beta}, g^{\alpha\beta})] = 1] \\ - \Pr[g, D \xleftarrow{\$} \mathbb{G}, \alpha, \beta \xleftarrow{\$} \mathbb{Z}_p : \mathcal{A}(\mathbb{G}, p, g, g^{\alpha}, g^{\beta}, D)] = 1]|\end{aligned}$$

The following lemma plays a crucial role in establishing the soundness of our proposed NIZK proof system.

Lemma 2.1. (Schwartz-Zippel Lemma) [43,47]. *Let $f(x_1, \ldots, x_n)$ be a non-zero polynomial of total degree d. Let $S \subseteq \mathbb{F}$ be any finite set. Then if $r_1, \ldots, r_n$ are randomly chosen from S, then* $\Pr[f(r_1, \ldots, r_n) = 0] \leq d/|S|$.

3 Dynamic TKEM with Transparent Setup

In this section, we introduce the definition and security models of a dynamic TKEM with transparent setup. Hereafter, for the sake of clarity, we will refer to it simply as a TKEM unless otherwise stated. We then propose our construction.

3.1 Definitions

We begin with the syntax of a TKEM. Our syntax is defined as a modification of dynamic TPKE [42], resulting in the TKEM concept. To ensure a transparent setup, no secret information is embedded in the **Setup** algorithm. Each user generates their own public and secret keys by running the **KeyGen** algorithm. The **Encap** algorithm produces a ciphertext header and a session key based on the specified threshold numbers t and n. Notably, an encapsulator (i.e., a user running the algorithm) can choose t and n in advance each time the algorithm is run, thereby reflecting the dynamic threshold setting. Due to the **CHVerify** algorithm, one can check if a given ciphertext header is well-formed or not. A legitimated user can decapsulate a given ciphertext header by running the **ShareDecaps** algorithm, which yields a (decapsulated) share. Each share can be verified through the **ShareVerify** algorithm. With a collection of t or more verified shares, the **Combine** algorithm recovers and outputs the session key. These algorithms are formally described as follows:

Definition 3.1 (Threshold Key Encapsulation Mechanism, TKEM)**.** A TKEM consists of the following six algorithms:

- **Setup**$(1^{\lambda}) \rightarrow PP$: The setup algorithm takes as input a security parameter λ and outputs the public parameters PP.
- **KeyGen**$(PP) \rightarrow (PK, SK)$: The key generation algorithm takes as input the public parameters PP, and it outputs a public key PK and a private key SK.
- **Encaps**$(PP, PL, t) \rightarrow (CH, K)$: The encapsulation algorithm takes as input the public parameters PP, a list of public keys $PL = (PK_1, \ldots, PK_n)$, and a threshold value t. It outputs a ciphertext header CH and a session key K.

- **CHVerify**$(PP, PL, CH) \rightarrow \{0,1\}$: The ciphertext header verify algorithm takes as input the public parameters PP, a list of public keys $PL = (PK_1, \ldots, PK_n)$, and a ciphertext header CH. It outputs 1 if a ciphertext header is valid and 0 otherwise.

- **ShareDecaps**$(PP, PL, CH, SK_i) \rightarrow \mu_i$: The share decapsulation algorithm takes as input the public parameters PP, a list of public keys $PL = (PK_1, \ldots, PK_n)$, a ciphertext header CH, and a private key SK_i. It outputs a decapsulated share μ_i.

- **ShareVerify**$(PP, CH, \mu_i, PK_i) \rightarrow \{0,1\}$: The share verification algorithm takes as input the public parameters PP, a ciphertext header CH, a decapsulated share μ_i, and a public key PK_i. It outputs 1 if the share is valid and 0 otherwise.

- **Combine**$(PP, PL, CH, \{\mu_i\}_{i \in S}) \rightarrow K$: The combining algorithm takes as input the public parameters PP, a list of public keys $PL = (PK_1, \ldots, PK_n)$, a ciphertext header CH, and a set of decapsulated shares $\{\mu_i\}_{i \in S}$. It outputs a session key K or $\perp$.

A TKEM is correct if the following conditions are satisfied: Let n and t be integers, both polynomially bounded in λ, such that $1 \leq t \leq n$. For any PP generated by **Setup**(1^λ), any $(PK_1, SK_1), \ldots, (PK_n, SK_n)$ generated by **KeyGen**(PP), and any (CH, K) generated by **Encaps**(PP, PL, t) for $PL = (PK_1, \ldots, PK_n)$, it is required that

- **CHVerify**$(PP, PL, CH) = 1$.

- For any $PK_i \in PL$, we have that $\mu_i =$ **ShareDecaps**(PP, PL, CH, SK_i) and **ShareVerify**$(PP, CH, \mu_i, PK_i) = 1$.

- For any $S = \{i_1, \ldots, i_t\} \subseteq [n]$ such that $|S| = t$ and $PK_{i_j} \in PL$, we have that $\{\mu_i =$ **ShareDecaps**$(PP, PL, CH, SK_i)\}_{i \in S}$ and **Combine**$(PP, PL, CH, \{\mu_i\}_{i \in S}) = K$.

Now, we consider the selective IND-CCA security of a TKEM, implying that the adversary cannot obtain any information about a session key corresponding to a given ciphertext header. In the security game, a challenger $\mathcal{C}$ provides an honest public key list to an adversary $\mathcal{A}$. Then $\mathcal{A}$ submits a corrupted public key list to $\mathcal{C}$. Moreover, $\mathcal{A}$ has access to a share decapsulation oracle, which enables to obtain a share of the ciphertext header. Since the security notion is selective, the adversary must submit a subset of corrupted public keys immediately after receiving the public parameters. These public keys must then be used in the challenge phase. The goal of $\mathcal{A}$ is to distinguish between the real session key and a randomly chosen key. Further details are as follows.

Definition 3.2. (SE-IND-CCA Security). The selective IND-CCA security game, denoted as $\mathbf{G}_{TKEM}^{SE\text{-}IND\text{-}CCA}$, between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ is demonstrated as follows:

- **Setup**: $\mathcal{C}$ obtains PP by running **Setup**(1^λ) and gives PP to $\mathcal{A}$. Then, $\mathcal{A}$ submits a threshold t^* and a public key list $PL_C^* = \{PK_1^*, \ldots, PK_{t^*-1}^*\}$ to $\mathcal{C}$. $\mathcal{C}$ generates $(PK_1, SK_1), \ldots, (PK_{n_1}, SK_{n_1})$ by running **KeyGen**(PP) and gives a set of honest users $\mathcal{U}_H = \{PK_1, \ldots, PK_{n_1}\}$ to $\mathcal{A}$. After then, $\mathcal{A}$ submits a set of corrupted users $\mathcal{U}_C = \{PK_{n_1+1}, \ldots, PK_{n_1+n_2}\}$ where $PL_C^* \subseteq \mathcal{U}_C$.

- **Query 1**: To obtain shares for honest users, $\mathcal{A}$ can make a share decapsulation query that includes a ciphertext header CH, a list of public keys $PL = \{PK_1', \ldots, PK_n'\} \subseteq \mathcal{U}_H \cup \mathcal{U}_C$, and a public key $PK_i \in \mathcal{U}_H$. If **CHVerify**$(PP, PL, CH) \neq 1$, then $\mathcal{C}$ outputs $\perp$. Otherwise, $\mathcal{C}$ returns a share μ_i that is obtained by running **ShareDecaps**(PP, PL, CH, SK_i) to $\mathcal{A}$.

- **Challenge**: $\mathcal{A}$ provides a list of public keys $PL_H^* = \{PK_{t^*}^*, \ldots, PK_{n^*}^*\} \subseteq \mathcal{U}_H$. At this point, a challenge public key list becomes $PL^* = PL_C^* \cup PL_H^*$. $\mathcal{C}$ selects a random K_0^* and obtains (CH^*, K_1^*) by running $\mathbf{Encaps}(PP, PL^*, t^*)$. Next, it chooses a random bit $b \in \{0,1\}$ and gives the challenge ciphertext tuple (CH^*, K_b^*) to $\mathcal{A}$.

- **Query 2**: $\mathcal{A}$ may additionally request share decapsulation queries and $\mathcal{C}$ handles these queries as in **Query 1**, except that $\mathcal{A}$ is not allowed to request a query for the challenge ciphertext header CH^*.

- **Guess**: Finally, $\mathcal{A}$ submits a bit $b' \in \{0,1\}$. $\mathcal{C}$ outputs 1 if $b = b'$. Otherwise, it outputs 0.

The advantage of $\mathcal{A}$ in the security parameter λ is defined as $\mathbf{Adv}_{TKEM}^{SE\text{-}IND\text{-}CCA}(\lambda) = \left| \Pr[\mathbf{G}_{TKEM}^{SE\text{-}IND\text{-}CCA} = 1] - \frac{1}{2} \right|$ where the probability is taken over all the randomness of the game. A TKEM is SE-IND-CCA secure if for any PPT adversary $\mathcal{A}$, $\mathbf{Adv}_{TKEM}^{SE\text{-}IND\text{-}CCA}(\lambda)$ is negligible.

Remark 1. Although we have proven the security of our TKEM in the selective IND-CCA security model, a stronger notion known as *adaptive security* exists. The main difference between the selective and the adaptive security games is that, in the adaptive setting, $\mathcal{A}$ does not submit t^* and PL_C^* during the setup phase. Instead, in the challenge phase, $\mathcal{A}$ submits $PL^* \subseteq \mathcal{U}_H \cup \mathcal{U}_C$ and a challenge threshold $t^* \in [n^*]$, such that $|PL^* \cap \mathcal{U}_C| \leq t^* - 1$. We will discuss the possibility of achieving this stronger notion in Section 6.

The decapsulation consistency (DC) is also a crucial security requirement of a TKEM. The DC security ensures that, for any valid shares generated from the same ciphertext header, their combination yields the same session key. Similar to the IND-CCA security, an adversary $\mathcal{A}$ against DC may compromise some public keys. Therefore, in the DC security game, $\mathcal{A}$ finally submits a ciphertext header and two distinct sets of shares derived from the header. The goal of $\mathcal{A}$ is that all shares in both sets are valid, and their combination results in different keys.

Definition 3.3 (DC Security)**.** The DC security game, denoted as $\mathbf{G}_{TKEM}^{DC}$, between a challenger $\mathcal{C}$ and an adversary $\mathcal{A}$ is demonstrated as follows:

- **Setup**: $\mathcal{C}$ obtains PP by running $\mathbf{Setup}(1^\lambda)$ and gives PP to $\mathcal{A}$. It generates key pairs $(PK_1, SK_1), \ldots, (PK_{n_1}, SK_{n_1})$ for honest users by running $\mathbf{KeyGen}(PP)$. It then gives $\mathcal{U}_H = \{PK_1, \ldots, PK_{n_1}\}$ to $\mathcal{A}$. Next, $\mathcal{A}$ submits $\mathcal{U}_C = \{PK_{n_1+1}, \ldots, PK_{n_1+n_2}\}$ of corrupted users.

- **Query**: To obtain shares for honest users, $\mathcal{A}$ can make a share decapsulation query that includes a ciphertext header CH, a list of public keys $PL = \{PK_1, \ldots, PK_n\} \subseteq \mathcal{U}_H \cup \mathcal{U}_C$, and a public key $PK_i \in \mathcal{U}_H$. Then $\mathcal{C}$ returns a share μ_i that is obtained by running $\mathbf{ShareDecaps}(PP, PL, CH, SK_i)$ to $\mathcal{A}$.

- **Forge**: Finally, $\mathcal{A}$ outputs a ciphertext header CH^* for a list of public keys $PL^* = \{PK_1^*, \ldots, PK_{n^*}^*\} \subseteq \mathcal{U}_H \cup \mathcal{U}_C$ and a threshold $t^* \in [n^*]$, and two sets of decapsulated shares $\{\mu_i\}_{i \in S}$ and $\{\mu'_j\}_{j \in S'}$ such that $t^* \leq |S| = |S'| \leq n^*$ and $S, S' \subseteq [n^*]$. $\mathcal{C}$ outputs 1 if the following three conditions are satisfied: (1) $\mathbf{Combine}(PP, PL^*, CH^*, \{\mu_i\}_{i \in S}) \neq \mathbf{Combine}(PP, PL^*, CH^*, \{\mu'_j\}_{j \in S'})$, (2) for all $i \in S$ and all $j \in S'$, $\mathbf{ShareVerify}(PP, CH^*, \mu_i, PK_i^*) = 1 \wedge \mathbf{ShareVerify}(PP, CH^*, \mu'_j, PK_j^*) = 1$, and (3) $\mathbf{CHVerify}(PP, PL^*, CH^*) = 1$. Otherwise, it outputs 0.

The advantage of $\mathcal{A}$ in the security parameter λ is defined as $\mathbf{Adv}_{TKEM}^{DC}(\lambda) = \Pr[\mathbf{G}_A^{DC} = 1]$ where the probability is taken over all the randomness of the game. A TKEM is DC secure if for any PPT adversary $\mathcal{A}$, the advantage of $\mathcal{A}$ is negligible.

3.2 Construction

3.2.1 High-level description

To construct our TKEM, we begin by combining the ElGamal PKE scheme and NIZK proof systems. Let the public key and the secret key of a user (indicated by idx_i) be $PK_i = X_i = g^{x_i}$ and $sk_i = x_i$, respectively. Due to the dynamic setting, an encapsulator chooses threshold values t and n to define a $(t-1)$-degree polynomial $f(\gamma) = \sum_{j=0}^{t-1} s_j \gamma^j$. Consider that each $f(idx_i)$ is a share for reconstructing the secret $s_0 = f(0)$. We have the following structure:

$$C = \left(C_1 = K \cdot g^{s_0}, C_2 = g^r, \{A_i = g^{f(idx_i)} \cdot X_i^r\}_{i \in [n]} \right), \tag{1}$$

where K is a session key hidden with g^{s_0} and the tuple (g^r, A_i) is an ElGamal ciphertext of $g^{f(idx_i)}$ under X_i. Then, the user with sk_i can recover $g^{f(idx_i)}$ through the ElGamal decryption process. Without loss of generality, given t values $g^{f(idx_1)}, \ldots, g^{f(idx_t)}$, the Lagrange interpolation can be used to reconstruct $g^{s_0} = \prod_{i \in [t]} (g^{f(idx_i)})^{L_i(0)}$. However, since the employed ElGamal scheme is not secure against IND-CCA, the structure of C inherits this weak security. We therefore apply the Naor-Yung transformation [38] to the encryption scheme, which is a classical solution for achieving CCA-security. This transformation results in the double key setting where $PK_i = (X_i = g^{x_i}, Y_i = g^{y_i})$ and $sk_i = (x_i, y_i)$. Consequently, the elements $\{B_i = g^{f(idx_i)} \cdot Y_i^r\}_{i \in [n]}$ are additionally included in C. Now, it is required to prove that C is well-formed. However, this is not straightforward due to the heterogeneous algebraic structure, which combines double ElGamal ciphertexts with a hidden session key. This structure makes it challenging to prove well-formedness. To address this, we adopt the approach of Mcfly [21] by letting $K = h^r$. This allows us to use NIZK-WF (see Chapter 3.3), an efficient Fiat-Shamir NIZK proof system that proves the same r is used in the exponent, thereby ensuring the well-formedness that we require. Note that the public parameter for this NIZK can be publicly generated, and hence, no trapdoor is embedded therein.

For additional security reasons, we require two NIZK proof systems: NIZK-PoK [11] that proves knowledge of discrete logarithms and NIZK-EDL [11] that proves the equality of discrete logarithms. Regarding IND-CCA, our security proof must use secret keys of corrupted users to generate a challenge ciphertext header, but these keys are committed and submitted by an adversary. Therefore, extractions of the secrets are necessary from proofs of knowledge. To be DC secure, each share (encrypted under PK_i) should be obtained by following the decryption with sk_i, and if so, $X_i^r = A_i / g^{f(idx_i)}$ can be computed correctly. This is ensured by checking the equality of the discrete logarithms of X_i and X_i^r on g and g^r, respectively.

3.2.2 Relations of the underlying NIZK proof systems

For clarity, we first present the relationships among the three underlying NIZK proof systems. Among them, the NIZK-PoK and NIZK-EDL proof systems rely on a well-known NIZK proof system for a generalized representation [11]. On the other hand, the NIZK-WF proof system is carefully constructed for our proposed TKEM; its details are provided in Section 3.3.

- The NIZK-PoK proof system is defined with respect to the following: Let the statement and witness be given by $s_{PoK} = (X, Y)$ and $w_{PoK} = (x, y)$, where $X, Y \in \mathbb{G}$ and $x, y \in \mathbb{Z}_p$. The relation $\mathcal{R}_{PoK}$ is then defined as:

  $$(s_{PoK}, w_{PoK}) \in \mathcal{R}_{PoK} \Leftrightarrow X = g^x \wedge Y = g^y,$$

  where g can be obtained from the CRS_{PoK}.

- The NIZK-WF proof system is defined with respect to the structure of $CL = (C_1, C_2, (A_1, B_1), \ldots, (A_n, B_n))$; therein, the notations used are consistent with those in Equation (1). Recall that each CL is generated based on a $(t-1)$-degree polynomial, leading to the parity check matrix $\mathbf{H} \in \mathbb{Z}_p^{(n-t+1)\times(n+1)}$ (derived from the Reed-Solomon codes). Let $(w_0, w_1, \ldots, w_n)^\top = \mathbf{v}^\top \cdot \mathbf{H}$ for $\mathbf{v} \xleftarrow{\$} \mathbb{Z}_p^{n-t+1}$ and let $s_{WF} = (CL, PK_1, \ldots, PK_n)$[1] and $w_{WF} = r$. The relation $\mathcal{R}_{WF}$ is then defined as:

$$(s_{WF}, w_{WF}) \in \mathcal{R}_{WF} \quad \Leftrightarrow \quad C_2 = g^r \wedge C_A = H_A^r \wedge C_B = H_B^r \wedge P_i = Q_i^r \ \ \forall i \in [n],$$

where

$$C_A = C_1^{w_0} \prod_{i=1}^{n} A_i^{w_i}, \quad C_B = C_1^{w_0} \prod_{i=1}^{n} B_i^{w_i},$$
$$H_A = h^{w_0} \prod_{i=1}^{n} X_i^{w_i}, \quad H_B = h^{w_0} \prod_{i=1}^{n} Y_i^{w_i},$$
$$P_i = \frac{A_i}{B_i}, \quad \text{and} \quad Q_i = \frac{X_i}{Y_i} \quad \forall i \in [n].$$

In the above, all group elements can be derived from the given CL and CRS_{WF}.

- The NIZK-EDL proof system is defined with respect to the following: Let the statement and witness be given by $s_{EDL} = (X_1, X_2, Y_1, Y_2)$ and $w_{EDL} = (x, y)$, where $X_1, X_2, Y_1, Y_2 \in \mathbb{G}$ and $x, y \in \mathbb{Z}_p$. The relation $\mathcal{R}_{EDL}$ is then defined as:

$$(s_{EDL}, w_{EDL}) \in \mathcal{R}_{EDL} \quad \Leftrightarrow \quad X_1 = g^x \wedge X_2 = g_2^x \wedge Y_1 = g^y \wedge Y_2 = g_2^y,$$

where $g, g_2 \in \mathbb{G}$ can be derived from CRS_{EDL}.

3.2.3 Detail Construction

Our TKEM is described as follows:

- **TKEM.Setup**(1^λ): It first generates a cyclic group $\mathbb{G}$ of prime order p with a random generator $g \in \mathbb{G}$. It selects a random element $h \in \mathbb{G}$. Next, it chooses four hash functions $H_0, H_1, H_2, H_4 : \{0,1\}^* \to \mathbb{Z}_p$ and $H_3 : \{0,1\}^* \to \mathbb{Z}_p^\ell$. Finally, it outputs public parameters $PP = (p, \mathbb{G}, g, h, H_0, H_1, H_2, H_3, H_4)$. We implicitly set $CRS_{PoK} = (p, \mathbb{G}, g, H_1)$ and $CRS_{WF} = (p, \mathbb{G}, g, h, H_2, H_3)$.

- **TKEM.KeyGen**(PP): It first selects random exponents $x, y \in \mathbb{Z}_p$ and computes $X = g^x$ and $Y = g^y$. It generates a proof $\pi =$ **NIZK-PoK.Prove**$^{H_1}(CRS_{PoK}, (X,Y), (x,y))$ for the proof of knowledge (x,y). Finally, it outputs a public key $PK = (X, Y, \pi)$ and a private key $SK = (x, y)$.

- **TKEM.Encaps**(PP, PL, t): Let $PL = \{PK_1, \ldots, PK_n\}$ where $PK_i = (X_i, Y_i, \pi_i)$. It proceeds as follows:

  1. For each $\pi_i \in PK_i$, it checks that **NIZK-PoK.Verify**$^{H_1}(CRS_{PoK}, (X,Y), \pi_i) = 1$. If one of these checks fails, it outputs $\perp$.

[1] For simplicity, we define this in an informal manner, while providing an intuitive description that CL is well-formed with respect to $PK_1, \ldots, PK_n$. Its formal statement will be provided in Section 3.3

2. It chooses random exponents $s_0, \ldots, s_{t-1} \in \mathbb{Z}_p$ and defines a $(t-1)$-degree polynomial $f(\gamma) = \sum_{j=0}^{t-1} s_j \cdot \gamma^j$ such that $f(0) = s_0$.
3. For $PK_i \in PL$, it calculates $H_0(PK_i) = idx_i$ and $f(idx_i)$.
4. It selects random exponent $r \in \mathbb{Z}_p$ and builds $CL = (C_1, C_2, (A_1, B_1), \ldots, (A_n, B_n), t)$ such that $C_1 = h^r g^{s_0}, C_2 = g^r$, $A_i = g^{f(idx_i)} X_i^r$, and $B_i = g^{f(idx_i)} Y_i^r$ for all $i \in [n]$.
5. Next, it generates a proof $\psi =$**NIZK-WF.Prove**$^{H_2}(CRS_{WF}, (CL, PK_1, \ldots, PK_n), r)$ for the well-formedness of CL.
6. Finally, it outputs a ciphertext header $CH = (CL, \psi)$ and a session key $K = h^r$.

- **TKEM.CHVerify**(PP, PL, CH): Let $PL = \{PK_1, \ldots, PK_n\}$ and $CH = (CL, \psi)$, where $CL = (C_1, C_2, (A_1, B_1), \ldots, (A_n, B_n), t)$. It outpus 1 if **NIZK-WF.Verify**$^{H_2}(CRS_{WF}, (CL, PK_1, \ldots, PK_n), \psi) = 1$ and 0 otherwise.

- **TKEM.ShareDecaps**(PP, PL, CH, SK_i): Let $PL = \{PK_1, \ldots, PK_n\}$ and $CH = (CL, \psi)$, where $CL = (C_1, C_2, (A_1, B_1), \ldots, (A_n, B_n), t)$, and $SK_i = (x_i, y_i)$. To decapsulate a share, it proceeds as follows:
  1. It implicitly sets $CRS_{EDL} = (p, \mathbb{G}, g, C_2, H_4)$.
  2. It computes $D_i = A_i \cdot (C_2)^{-x_i}$ and generates a proof $\sigma_i =$ **NIZK-EDL.Prove**$^{H_4}(CRS_{EDL}, (X_i, A_i/D_i, Y_i, B_i/D_i), (x_i, y_i))$ for the equality of discrete logarithms.
  3. Finally, it outputs a decapsulated share $\mu_i = (D_i, \sigma_i)$.

- **TKEM.ShareVerify**(PP, CH, μ_i, PK_i): Let $CH = (CL, \psi)$, where $CL = (C_1, C_2, (A_1, B_1), \ldots, (A_n, B_n), t)$, $\mu_i = (D_i, \sigma_i)$, and $PK_i = (X_i, Y_i, \pi_i)$. It implicitly sets $CRS_{EDL} = (p, \mathbb{G}, g, C_2, H_4)$. If **NIZK-EDL.Verify**$^{H_4}(CRS_{EDL}, (X_i, A_i/D_i, Y_i, B_i/D_i), \sigma_i) = 1$, then outputs 1. Otherwise, it outputs 0.

- **TKEM.Combine**$(PP, PL, CH, \{\mu_i\}_{i \in S})$: Let $PL = \{PK_1, \ldots, PK_n\}$ and $CH = (CL, \psi)$, where $CL = (C_1, C_2, (A_1, B_1), \ldots, (A_n, B_n), t)$, $\mu_i = (D_i, \sigma_i)$, and $|S| = t$. It computes $K = C_1 \cdot \prod_{j \in S} D_j^{-L_j(0)}$ where $L_j(0) = \prod_{k \in S, k \neq j} (-idx_k)/(idx_j - idx_k)$ is a Lagrange coefficient. It outputs a session key K.

Correctness. The correctness of our TKEM is given by the following equation:

$$\begin{aligned} C_1 \cdot \prod_{j \in [t]} D_j^{-L_j(0)} &= h^r g^{f(0)} \cdot \prod_{j \in S} (g^{f(idx_j)})^{-L_j(0)} \\ &= h^r g^{f(0)} \cdot g^{\sum_{j \in S} -f(idx_j) L_j(0)} \\ &= h^r g^{f(0)} \cdot g^{-f(0)} = h^r = K. \end{aligned}$$

Remark 2. Note that this setup algorithm can be **transparent** by ensuring that such generations do not require any secret information. Specifically, the generation for $(g, \mathbb{G})$ is deterministic and public, and the element h can be generated by a map-to-point hash function. This can be publicly verified given the specification of the function.

Remark 3. Since our TKEM supports a full dynamic property, the threshold parameters (t, n) are determined by the encapsulator. In practice, the output lengths of H_3 and a parity check matrix $\mathbf{H}$ are decided based on (t, n). Furthermore, both the verifier and the decapsulator can use (t, n) to generate the same H_3 and $\mathbf{H}$ as used by the encapsulator.

3.3 NIZK for Well-Formedness of Ciphertext Header

We describe our NIZK construction for well-formedness of ciphertext header, referred to as NIZK-WF. Our NIZK-WF system is a modification of [21] eliminating the need for pairings. In the following description, we will show that the NIZK-WF proof system satisfies all properties outlined in Section 2.4, and the notations used retain the same meanings as previously defined.

Let a list of ciphertext elements be denoted as $CL = (C_1, C_2, (A_1, B_1), \ldots, (A_n, B_n), t)$ such that $C_1 = h^r g^{f(0)}, C_2 = g^r, A_i = g^{f(idx_i)} X_i^r$, and $B_i = g^{f(idx_i)} Y_i^r$. We say that CL is well-formed for our TKEM if and only if the following conditions are satisfied:

- The same exponent r is used for all elements in CL.
- For all $i \in [n]$, both A_i and B_i are ElGamal ciphertexts of the same $f(idx_i)$.
- Each $f(idx_i)$ is a valid secret share of the secret $f(0)$.

Fortunately, the conditions 1 and 2 might be handled with the existing techniques for NIZK-EDL and for the double ElGamal encryption with the shared randomness, respectively. We thus focus on the final condition: By the Reed-Solomon (RS) codes, anyone given $(0, idx_1, \ldots, idx_n)$ can obtain a parity check matrix $\mathbf{H} \in \mathbb{Z}_p^{(n-t+1)\times(n+1)}$ such that $\mathbf{H} \cdot \mathbf{f} = \mathbf{0}$, where $\mathbf{f} = (f(0), f(idx_1), \ldots, f(idx_n))^\top$. Assume that a vector $\mathbf{v} \in \mathbb{Z}_p^{n-t+1}$ has the uniformly random distribution via the random oracle. The fact that $\mathbf{v}^\top \cdot \mathbf{H} \cdot \mathbf{f} = \mathbf{v}^\top \cdot \mathbf{0} = 0$ implies that secret shares are all valid with overwhelming probability. Based on these, there is a way to check whether $\mathbf{f}$ is composed of valid secret shares: From CL and $\mathbf{w} = (w_0, \ldots, w_n)^\top$, we can derive the following components:

$$
\begin{aligned}
C_A &= C_1^{w_0} \prod_{i=1}^{n} A_i^{w_i},\ C_B = C_1^{w_0} \prod_{i=1}^{n} B_i^{w_i}, \\
H_A &= h^{w_0} \prod_{i=1}^{n} X_i^{w_i},\ H_B = h^{w_0} \prod_{i=1}^{n} Y_i^{w_i}.
\end{aligned} \tag{2}
$$

Assume that h^r in C_1, X_i^r in A_i, and Y_i^r in Y_i are well-formed, which partially satisfies the condition 1 above. We then have the following relations:

$$
\begin{aligned}
C_A &= (h^r g^{f(0)})^{w_0} \prod_{i=1}^{n} (g^{f(idx_i)} X_i^r)^{w_i} \\
&= (h^{w_0} \prod_{i=1}^{n} X_i^{w_i})^r g^{f(0)w_0 + \sum_{i=1}^{n} f(idx_i) w_i} = H_A^r \cdot g^{\mathbf{w}^\top \mathbf{f}}, \\
C_B &= (h^r g^{f(0)})^{w_0} \prod_{i=1}^{n} (g^{f(idx_i)} Y_i^r)^{w_i} \\
&= (h^{w_0} \prod_{i=1}^{n} Y_i^{w_i})^r g^{f(0)w_0 + \sum_{i=1}^{n} f(idx_i) w_i} = H_B^r \cdot g^{\mathbf{w}^\top \mathbf{f}}.
\end{aligned}
$$

If we set $\mathbf{w}^\top = \mathbf{v}^\top \cdot \mathbf{H}$ then $\mathbf{w}^\top \cdot \mathbf{f} = \mathbf{v}^\top \cdot \mathbf{H} \cdot \mathbf{f} = 0$, which results in $C_A = H_A^r \wedge C_B = H_B^r$. Let $P_i = A_i / B_i$ and $Q_i = X_i / Y_i$ for all $i \in [n]$. The remaining condition (including the condition 2) is now ensured by $C_2 = g^r \wedge P_i = Q_i^r$ for the same r. Consequently, we only need to verify that the exponent r is consistently used for g, H_A, H_B, and P_i, thereby completing the verification of all conditions simultaneously. To formally prove this argument, we provide the following lemma.

Lemma 3.1. *Let $CL = (C_1, C_2, (A_1, B_1), \ldots (A_n, B_n), t)$ be a list of ciphertext elements. If there exists an exponent r such that $(C_2 = g^r \wedge C_A = H_A^r \wedge C_B = H_B^r \wedge P_1 = Q_1^r \wedge \ldots \wedge P_n = Q_n^r)$, then CL is well-formed with probability of at least $1 - 3/p$.*

Proof. We demonstrate that if CL is not well-formed, then such an exponent r exists with negligible probability. Consider that CL is defined as $\left(h^{r_0} g^{f_0(0)}, g^r, (g^{f_1(idx_1)} X_1^{r_1}, g^{f_1'(idx_1)} Y_1^{r_1'}), \ldots, (g^{f_n(idx_n)} X_n^{r_n}, g^{f_n'(idx_n)} Y_n^{r_n'}), t\right)$. If, for all $i \in [n]$, the event E_1 where $r = r_i = r_i'$ and the event E_2 where $f_0 = f_i = f_i'$ occur together, CL is considered well-formed. Hence, when CL is not well-formed, there are three distinct cases: *(Case 1)* $\neg E_1 \wedge \neg E_2$, *(Case 2)* $\neg E_1 \wedge E_2$, and *(Case 3)* $E_1 \wedge \neg E_2$.

In Case 1, from the equations in (2), the equation $\log_g C_A - \log_g H_A^r = 0$ gives the following:

$$\begin{aligned}\log_g C_A - \log_g H_A^r &= \log_g h \cdot w_0 \cdot r_0 + w_0 \cdot f_0(0) + \sum_{i=1}^{n} w_i (x_i r_i + f_i(idx_i)) - (\log_g h \cdot w_0 + \sum_{i=1}^{n} w_i x_i) \cdot r \\ &= (\log_g h \cdot r_0 + f_0(0) - \log_g h \cdot r) \cdot w_0 + \sum_{i=1}^{n} (x_i r_i + f_i(idx_i) - x_i r) \cdot w_i = 0. \end{aligned} \tag{3}$$

Similarly, we have the following equation:

$$\log_g C_b - \log_g H_b^r = (\log_g h \cdot r_0 + f_0(0) - \log_g h \cdot r) \cdot w_0 + \sum_{i=1}^{n} (y_i r_i' + f_i'(idx_i) - y_i r) \cdot w_i = 0. \tag{4}$$

To achieve $C_A = H_A^r$ and $C_b = H_b^r$ without controlling the random oracle H_3, all of terms in (3) and (4) must be zero. This is because $\mathbf{w} = (w_0, \ldots, w_n)^\top$ was determined by $\mathbf{v} = H_3(CL)$. However, in this way, CL has the structure $(h^r, g^r, \{X_i^r, Y_i^r\}, t)$, which is still well-formed with respect to the zero polynomial (i.e., all coefficients are zero). This contradicts the assumption that CL is not well-formed. Hence, there must be at least one non-zero term in (3). This implies that $C_A = H_A^r$ holds with a probability of at most $1/p$, due to the Schwartz-Zippel lemma (described in Section 2.1).

In Case 2, we can have $f_0(0) w_0 + \sum_{i=1}^{n} f_i(idx_i) w_i = 0$, i.e., $\mathbf{w}^\top \cdot \mathbf{f} = 0$, where $\mathbf{f} = (f_0(0), f_1(idx_1), \ldots, f_n(idx_n))$. Hence, the equation (3) can be simplified as $(\log_g h \cdot r_0 - \log_g h \cdot r) \cdot w_0 + \sum_{i=1}^{n} (x_i r_i - x_i r) \cdot w_i = 0$, where at least one non-zero term exists, because E_1 does not happen. Therefore, $C_A = H_A^r$ holds with the probability of at most $1/p$, due to the Schwartz-Zippel lemma.

In Case 3, assuming that $P_i = A_i / B_i = (X_i / Y_i)^r = Q_i^r$, we have $f_i(idx_i) = f_i'(idx_i)$ as $\log_g P_i - \log_g Q_i^r = (f_i(idx_i) + x_i r_i - f_i'(idx_i) - y_i r_i') - r(x_i - y_i) = f_i(idx_i) - f_i'(idx_i) = 0$. From this, there must exist some i such that $f_0 \neq f_i$ of $\mathbf{f}$ (defined as before). Let $\mathbf{H}$ be a parity check matrix of the RS codes, with respect to $\mathbf{f}' = (f(0), f(idx_1), \ldots, f(idx_n))^\top$, where $f = f_0$. Then $C_A = H_A^r$ results in $\mathbf{w}^\top \cdot \mathbf{f} = \mathbf{v}^\top \cdot \mathbf{H} \cdot \mathbf{f} = 0$ with $\mathbf{f} \neq \mathbf{f}'$. However, this holds when the randomly sampled $\mathbf{v}$ unfortunately leads to this equality. Due to the Schwartz-Zippel lemma, the probability that such a bad $\mathbf{v}$ is sampled is at most $1/p$. □

Based on these, we formally define $s_{WF} = (C_2 = g^r, C_A = H_A^r, C_B = H_B^r, P_1 = Q_1^r, \ldots, P_n = Q_n^r)$ and $w_{WF} = r$. As described earlier, for simplicity in our TKEM explanation, we also define the alternative statement informally as $s_{WF}' = (CL, PK_1, \ldots, PK_n)$. Accordingly, the following proving and verification algorithms of the NIZK-NF proof system start by computing C_A, C_B, and $\{P_i, Q_i\}$ as follows:

- **NIZK-WF.Prove**$^{H_2}(CRS_{WF}, s_{WF}', w_{WF})$: Let $CRS_{WF} = (p, \mathbb{G}, g, h, H_2, H_3)$, $CL = (C_1, C_2, (A_1, B_1), \ldots, (A_n, B_n), t) \in s_{WF}'$, and $PK_i = (X_i, Y_i, \pi_i) \in s_{WF}'$. To generate a proof, it proceeds as follow:

  1. It prepares a parity check matrix $\mathbf{H} \in \mathbb{Z}_p^{(n-t+1)\times(n+1)}$ of the RS codes. Next, it obtains a random vector $\mathbf{v} = H_3(CL, n-t+1) \in \mathbb{Z}_p^{n-t+1}$ and computes $\mathbf{w}^\top = \mathbf{v}^\top \cdot \mathbf{H}$ where $\mathbf{w} = (w_0, \ldots, w_n)^\top$.

2. It computes $C_A = C_1^{w_0} \prod_{i=1}^n A_i^{w_i}$, $C_B = C_1^{w_0} \prod_{i=1}^n B_i^{w_i}$, $H_A = h^{w_0} \prod_{i=1}^n X_i^{w_i}$, and $H_B = h^{w_0} \prod_{i=1}^n Y_i^{w_i}$.
3. It computes $P_i = A_i/B_i$ and $Q_i = X_i/Y_i$ for $i \in [n]$.
4. It selects random $u \in \mathbb{Z}_p$ and sets $U = g^u, U_A = H_A^u, U_B = H_B^u$, and $U_i = Q_i^u$ for $i \in [n]$.
5. And it obtains $e = H_2(CL, C_A, C_B, H_A, H_B, (P_1, Q_1), \ldots, (P_n, Q_n), U, U_A, U_B, U_1, \ldots, U_n)$.
6. It calculates $z = u + er$ and outputs a proof $\psi = (e, z)$.

- **NIZK-WF.Verify**$^{H_2}(CRS_{WF}, s'_{WF}, \psi)$: Let $CL = (C_1, C_2, (A_1, B_1), \ldots, (A_n, B_n), t)$, $PK_i = (X_i, Y_i, \pi_i)$, and $\psi = (e, z)$. It proceeds as follows;

1. It prepares a parity check matrix $\mathbf{H} \in \mathbb{Z}_p^{(n-t+1)\times(n+1)}$ of the RS codes. It obtains a random vector $\mathbf{v} = H_3(CL, n-t+1) \in \mathbb{Z}_p^{n-t+1}$ and computes $\mathbf{w}^\top = \mathbf{v}^\top \cdot \mathbf{H}$ where $\mathbf{w} = (w_0, \ldots, w_n)^\top$.
2. It computes $C_A = C_1^{w_0} \prod_{i=1}^n A_i^{w_i}$, $C_B = C_1^{w_0} \prod_{i=1}^n B_i^{w_i}$, $H_A = h^{w_0} \prod_{i=1}^n X_i^{w_i}$, and $H_B = h^{w_0} \prod_{i=1}^n Y_i^{w_i}$.
3. It computes $P_i = A_i/B_i$ and $Q_i = X_i/Y_i$ for $i \in [n]$.
4. It computes $U' = g^z \cdot C_2^{-e}$, $U'_A = H_A^z \cdot C_A^{-e}, U'_B = H_B^z \cdot C_B^{-e}$, and $U'_i = Q_i^z \cdot P_i^{-e}$ for $i \in [n]$.
5. It checks $e \stackrel{?}{=} H_2(CL, C_A, C_B, H_A, H_B, (P_1, Q_1), \ldots, (P_n, Q_n), U', U'_A, U'_B, U'_1, \ldots, U'_n)$.
6. If the check succeeds, it outputs 1. Otherwise, it outputs 0.

Completeness. Let $\mathbf{f} = (f(0), f(idx_1), \ldots, f(idx_n))^\top$ be a vector of polynomial values that are honestly generated and embedded in ciphertext elements CL. We have $\mathbf{v}^\top \cdot \mathbf{H} \cdot \mathbf{f} = \mathbf{w}^\top \cdot \mathbf{f} = f(0)w_0 + \sum_{i=1}^n f(idx_i)w_i = 0$ since $\mathbf{H} \cdot \mathbf{f} = 0$ where $\mathbf{H}$ is the parity check matrix. Thus, we obtain the following equations

$$\begin{aligned}
C_A &= C_1^{w_0} \prod_{i=1}^n A_i^{w_i} = \left(h^r g^{f(0)}\right)^{w_0} \prod_{i=1}^n \left(g^{f(idx_i)} X_i^r\right)^{w_i} \\
&= \left(h^{w_0} \prod_{i=1}^n X_i^{w_i}\right)^r g^{f(0)w_0 + \sum_{i=1}^n f(idx_i)w_i} = H_A^r, \\
C_B &= C_1^{w_0} \prod_{i=1}^n B_i^{w_i} = \left(h^r g^{f(0)}\right)^{w_0} \prod_{i=1}^n \left(g^{f(idx_i)} Y_i^r\right)^{w_i} \\
&= \left(h^{w_0} \prod_{i=1}^n Y_i^{w_i}\right)^r g^{f(0)w_0 + \sum_{i=1}^n f(idx_i)w_i} = H_B^r, \\
P_i &= A_i/B_i = (g^{f(idx_i)X_i^r})/(g^{f(idx_i)Y_i^r}) = (X_i/Y_i)^r = Q_i^r \text{ for } i \in [n].
\end{aligned}$$

If $e = H_2(CL, C_A, C_B, H_A, H_B, (Q_1, P_1), \ldots, (Q_n, P_n), U, U_A, U_B, U_1, \ldots, U_n)$ and $z = u + er$, then it is easy to check that the following verification equations are satisfied as $g^z = g^u \cdot (g^r)^e = U \cdot C_2^e$, $H_A^z = H_A^u \cdot (H_A^r)^e = U_A \cdot C_A^e$, $H_B^z = H_B^u \cdot (H_B^r)^e = U_A \cdot C_B^e$, $Q_i^z = Q_i^u \cdot (Q_i^r)^e = U_i \cdot ((X_i/Y_i)^r)^e = U_i \cdot (P_i)^e$ for $i \in [n]$.

(Simulation) Soundness. Due to Lemma 3.1, the NIZK-WF proof system essentially proves the equality of discrete logarithms, as in the NIZK-EDL proof system. Therefore, it is clear that this proof system guarantees the (simulation) soundness property.

Zero-Knowledge. This property can be easily achieved for a simulated proof $\psi = (U, U_A, U_B, U_1, \ldots, U_n, z)$ if a simulator first selects random exponents $e, z \in \mathbb{Z}_p$ and sets elements $U = g^z \cdot C_2^{-e}$, $U_A = H_A^z \cdot C_A^{-e}$, $U_B = H_B^z \cdot C_B^{-e}$ and $U_i = Q_i^z \cdot P_i^{-e}$ for $i \in [n]$ by programming the random oracle. This simulated proof is identically distributed to the real one.

Table 2: Differences between Hybrid games from $\mathbf{G}_0$ to $\mathbf{G}_5$

Game	Session Key		Ciphertext Header				NIZK	Decap.	Witness
	K_0^*	K_1^*	C_1^*	$B_{t^*}^*$	…	$B_{n^*}^*$		Keys	Ext.
$\mathbf{G}_0$	R	h^r	$h^r g^{f(0)}$	$g^{f(idx_{t^*})} Y_{t^*}^r$	…	$g^{f(idx_{n^*})} Y_{n^*}^r$	*Real*	x_i	no
$\mathbf{G}_1$	R	h^r	$h^r g^{f(0)}$	$g^{f(idx_{t^*})} Y_{t^*}^r$	…	$g^{f(idx_{n^*})} Y_{n^*}^r$	*Real*	x_i	yes
$\mathbf{G}_2$	R	h^r	$h^r g^{f(0)}$	$g^{f(idx_{t^*})} Y_{t^*}^r$	…	$g^{f(idx_{n^*})} Y_{n^*}^r$	*Sim*	x_i	yes
$\mathbf{G}_3$	R	h^r	$h^r g^{f(0)}$	R_{t^*}	…	R_{n^*}	*Sim*	x_i	yes
$\mathbf{G}_4$	R	h^r	$h^r g^{f(0)}$	R_{t^*}	…	R_{n^*}	*Sim*	y_i	yes
$\mathbf{G}_5$	R	h^r	R_1	R_{t^*}	…	R_{n^*}	*Sim*	y_i	yes
$\mathbf{G}_6$	R	R_0	R_1	R_{t^*}	…	R_{n^*}	*Sim*	y_i	yes

In our SE-IND-CCA security proof, a challenge ciphertext header $CH^* = ((C_1^*, C_2^*, (A_1^*, B_1^*), \ldots, (A_{n^*}^*, B_{n^*}^*), t^*), \psi^*)$ related a session key K_b^* ($b \in \{0,1\}$) changes from game $\mathbf{G}_0$ to $\mathbf{G}_5$, with the varying components indicated by grey boxes. Each of $R, R_0, \ldots R_{n^*}$ denotes random replacement. In the NIZK column, *Real* indicates that all NIZK proofs given to an adversary are generated as real ones, while *Sim* indicates that they are generated by a zero-knowledge simulator. The Decap.Keys column represents the secret key used to respond to share decapsulation queries. The last column shows whether straight-line extractions were successfully carried out on proofs $\pi_1, ..., \pi_{t^*-1}$ with respect to PL_C^* (initially submitted by the adversary). Components not shown in the table remain unchanged.

4 Security Analysis

In this section, we prove that our TKEM, proposed in Section 3.2, guarantees the SE-IND-CCA security and the DC security.

Theorem 4.1. *The proposed TKEM is SE-IND-CCA secure if the DDH assumption holds, the underlying NIZK proof systems are zero-knowledge, the NIZK-PoK is extractable with negligible error, and the NIZK-WF is simulation sound, in the random oracle model.*

Proof. To prove the SE-IND-CCA security of our TKEM, we define a sequence of hybrid games $\mathbf{G}_0, \mathbf{G}_1, \ldots, \mathbf{G}_6$. As shown in Table 2, $\mathbf{G}_0$ is the original game defined in Definition 3.2, where a PPT adversary $\mathcal{A}$ aims to distinguish which session key from $\{K_b^*\}_{b\in\{0,1\}}$ is associated with a given ciphertext header CH^*.

Intuitively, our hybrid games transition from a setting in which CH^* directly encodes the challenge key (Game $\mathbf{G}_0$) to one in which the ciphertext header is entirely independent of both candidate keys (Game $\mathbf{G}_6$). Each step $\mathbf{G}_i \to \mathbf{G}_{i+1}$ ($i \in [0,5]$) introduces minimal changes that are indistinguishable to $\mathcal{A}$, unless an underlying cryptographic assumption is broken. The transitions proceed as follows: In $\mathbf{G}_1$, the challenger $\mathcal{C}$ extracts witnesses from NIZK-PoK proofs of honest users and stores them for later use; In $\mathbf{G}_2$, $\mathcal{C}$ replaces all NIZK proofs with simulated ones, as honest secret keys are no longer assumed to be known in subsequent games; In $\mathbf{G}_3$, $\mathcal{C}$ modifies components of the ElGamal ciphertexts in CH^* to eliminate dependencies between each ciphertext component; In $\mathbf{G}_4$, $\mathcal{C}$ switches to using an alternative secret key to correctly respond to decapsulation queries, since the original key may no longer be known; In $\mathbf{G}_5$, $\mathcal{C}$ alters another component of CH^* to make the encoded K_b^* independent of the remaining components; and in $\mathbf{G}_6$, $\mathcal{C}$ replaces CH^* entirely with values independent of both K_0^* and K_1^*. In the final game, the ciphertext header contains no information

about the challenge bit. Hence, the adversary's view is independent of the bit b, and its success probability is exactly $1/2$, establishing SE-IND-CCA security.

Formally, we define our hybrid game as follows. Let CH^* consist of $((C_1^*, C_2^*, \{A_i^*, B_i^*\}_{i=1}^{n^*}, t^*), \psi^*)$ and let $\mathbf{S}_i$ denote the event that $\mathbf{G}_i$ outputs 1. The detailed descriptions of the hybrid games are given below:

- Game $\mathbf{G}_0$: This game $\mathbf{G}_0$ is the original game. Thus, we obtain the following equation

$$\mathbf{Adv}_{TKEM}^{SE\text{-}IND\text{-}CCA}(\lambda) = \big|\Pr[\mathbf{S}_0] - 1/2\big|.$$

- Game $\mathbf{G}_1$: This game $\mathbf{G}_1$ is the same as $\mathbf{G}_0$ except that, after $\mathcal{A}$ submits $PL_C^* = \{PK_1^*, ..., PK_{t^*-1}^*\}$, the challenger $\mathcal{C}$ parses each PK_i^* as (X_i, Y_i, π_i) for $i \in [t^* - 1]$. It then runs the straight-line extractor $\mathcal{E}$ on each π_i. If any extraction fails, $\mathcal{C}$ aborts. Since each extraction fails with probability at most $\varepsilon_{\mathcal{E}}$, we obtain the following bound:

$$\big|\Pr[\mathbf{S}_1] - \Pr[\mathbf{S}_0]\big| \leq (t^* - 1) \cdot \varepsilon_{\mathcal{E}}$$

- Game $\mathbf{G}_2$: This game $\mathbf{G}_2$ is the same as $\mathbf{G}_1$ except that all NIZK proofs are replaced by simulated proofs that are generated without the corresponding witness. By the zero-knowledge property of each NIZK proof system, we obtain the following inequality

$$\begin{aligned}\big|\Pr[\mathbf{S}_2] - \Pr[\mathbf{S}_1]\big| \leq\ & n_1 \cdot \mathbf{Adv}_{NIZK\text{-}PoK}^{ZK}(\lambda) \\ & + \mathbf{Adv}_{NIZK\text{-}WF}^{ZK}(\lambda) + q_D \cdot \mathbf{Adv}_{NIZK\text{-}EDL}^{ZK}(\lambda),\end{aligned}$$

  where n_1 is the number of honest users and q_{D} is the total number of share decapsulation queries.

- Game $\mathbf{G}_3$: This game $\mathbf{G}_3$ is the same as $\mathbf{G}_2$ except that $\{B_i^*\}_{i \in [t^*, n^*]}$ in CH^* are replaced by $\{R_i\}_{i \in [t^*, n^*]}$, respectively, where $R_i \xleftarrow{\$} \mathbb{G}$. From Lemma 4.2, we have the following inequality

$$\big|\Pr[\mathbf{S}_3] - \Pr[\mathbf{S}_2]\big| \leq \mathbf{Adv}_{DDH}(\lambda).$$

- Game $\mathbf{G}_4$: This game $\mathbf{G}_4$ is the same as $\mathbf{G}_3$ except for handling a share decapsulation query of (CH, PL, PK_i). To respond this query, CH is now decapsulated by using $y_i \in SK_i$, instead of x_i. This modification is detectable if $\mathcal{A}$ creates a particular $CH = (\ldots, (A_i, B_i), \ldots)$ where B_i does not mirror A_i, but passes the **CHVerify** algorithm. However, this is possible with negligible probability due to the (simulation) soundness property of NIZK-WF. Hence, we have the following inequality

$$\big|\Pr[\mathbf{S}_4] - \Pr[\mathbf{S}_3]\big| \leq \mathbf{Adv}_{NIZK\text{-}WF}^{S-Sound}(\lambda).$$

- Game $\mathbf{G}_5$: This game $\mathbf{G}_5$ the same as $\mathbf{G}_4$ except that C_1^* in CH^* is replaced by $R_1 \xleftarrow{\$} \mathbb{G}$. From Lemma 4.3, we obtain the following inequality

$$\big|\Pr[\mathbf{S}_5] - \Pr[\mathbf{S}_4]\big| \leq \mathbf{Adv}_{DDH}(\lambda).$$

- Game $\mathbf{G}_6$: This game $\mathbf{G}_6$ is the same as $\mathbf{G}_5$ except that K_1^* is set to $R_0 \xleftarrow{\$} \mathbb{G}$. From Lemma 4.4, we obtain the following inequality

$$\big|\Pr[\mathbf{S}_6] - \Pr[\mathbf{S}_5]\big| \leq \mathbf{Adv}_{DDH}(\lambda).$$

  In this final game $\mathbf{G}_6$, the advantage of $\mathcal{A}$ is zero as CH^* is now generated completely independent of both K_0^* and K_1^*; thus, we have $\Pr[\mathbf{S}_6] - 1/2 = 0$.

By combining all together, we can obtain the following inequality that bounds the advantage of $\mathcal{A}$ as

$$\begin{aligned}\mathbf{Adv}_{TKEM}^{SE\text{-}IND\text{-}CCA}(\lambda) &= \left|\Pr[\mathbf{S}_0] - 1/2\right| \\ &\leq \sum_{k=1}^{6} \left|\Pr[\mathbf{S}_{k-1}] - \Pr[\mathbf{S}_k]\right| + \left|\Pr[\mathbf{S}_6] - 1/2\right| \\ &\leq n_1 \cdot \mathbf{Adv}_{NIZK\text{-}PoK}^{ZK}(\lambda) + \mathbf{Adv}_{NIZK\text{-}WF}^{ZK}(\lambda) + q_D \cdot \mathbf{Adv}_{NIZK\text{-}EDL}^{ZK}(\lambda) \\ &\quad + \mathbf{Adv}_{NIZK\text{-}WF}^{S-Sound}(\lambda) + 3\mathbf{Adv}_{DDH}(\lambda) + (t^* - 1) \cdot \varepsilon_{\mathcal{E}}.\end{aligned}$$

This completes the proof. □

Lemma 4.2. *If the DDH assumption holds, then no PPT algorithm can distinguish between $\boldsymbol{G}_2$ and $\boldsymbol{G}_3$.*

Proof. Given a DDH tuple $(g, \hat{g}_1 = g^{\alpha}, \hat{g}_2 = g^{\beta}, D)$, we can construct a reduction $\mathcal{R}$ that serves as a challenger to an adversary $\mathcal{A}$ in the SE-IND-CCA security game. $\mathcal{R}$ interacts with $\mathcal{A}$ as follows:

Setup: $\mathcal{R}$ randomly chooses $\hat{r} \in \mathbb{Z}_p$ to compute $h = g^{\hat{r}}$. $\mathcal{R}$ generates public parameters $PP = (p, \mathbb{G}, g, h, H_0, H_1, H_2, H_3, H_4)$ as in the real scheme, except that these hash functions are modeled as random oracles. After $\mathcal{R}$ sends PP to $\mathcal{A}$, the adversary responds with t^* and PL_C^* such that $PL_C^* = \{PK_1^*, \ldots, PK_{t^*-1}^*\}$. For each $PK_j^* = (X_j, Y_j, \pi_j) \in PL_C^*$, $\mathcal{R}$ successfully extracts and stores the witness (x_j, y_j) corresponding to π_j. Then $\mathcal{R}$ sets $\mathcal{U}_H$ as empty, and generates each public key PK_i of honest users as follows:

1. It chooses $x_i, \rho_{i,1}, \rho_{i,2} \in \mathbb{Z}_p$ randomly and stores them to deal with the challenge phase.
2. It computes $X_i = g^{x_i}$ and $Y_i = \hat{g}_1^{\rho_{i,1}} g^{\rho_{i,2}}$.
3. It simulates a proof π_i of NIZK-PoK.
4. It adds $PK_i = (X_i, Y_i, \pi_i)$ to $\mathcal{U}_H$.

Clearly, $|\mathcal{U}_H|$ is polynomially bounded. During the simulation, $y_i \in SK_i$ of an honest user is implicitly set to $y_i = \alpha\rho_{i,1} + \rho_{i,2}$, where a is unknown to $\mathcal{R}$. Since $\rho_{i,1}$ and $\rho_{i,2}$ are both chosen randomly and independently of $\mathcal{A}$, this y_i is indistinguishable from the real one.

Query 1: The hash oracles are managed by returning a random value for a new input, without simulating NIZK proofs. When $\mathcal{A}$ requests a share decapsulation query for (PL, CH, PK_i), where $\mathbf{CHVerify}(PP, PL, CH) = 1$ and $PK_i \in \mathcal{U}_H$, $\mathcal{R}$ returns $\mu_i = \mathbf{ShareDecaps}(PP, PL, CH, SK_i)$ using $x_i \in SK_i$, as a response.

Challenge: $\mathcal{A}$ first sends $PL_H^* \in \mathcal{U}_H$ and t^*. Let $PL^* = PL_C^* \cup PL_H^*$ be rewritten by $\{PK_1^*, \ldots, PK_{n^*}^*\}$. For all PK_i^*, $\mathcal{R}$ obtains $H_0(PK_i^*) = idx_i^*$. $\mathcal{R}$ then randomly picks $s_0, \ldots, s_{t^*-1} \in \mathbb{Z}_p$ and generates a $(t^* - 1)$-degree polynomial $f(\gamma) = \sum_{j=0}^{t^*-1} s_j \gamma^j$. The challenge ciphertext header $CH^* = (C_1^*, C_2^*, (A_1^*, B_1^*), \ldots, (A_{n^*}^*, B_{n^*}^*), t^*, \psi^*)$ is generated as follows:

1. It computes $C_1^* = \hat{g}_2^{\hat{r}} g^{s_0}$ and $C_2^* = \hat{g}_2$.
2. For $i \in [n^*]$, it computes $A_i^* = g^{f(idx_i^*)} \hat{g}_2^{x_i}$.
3. For $i \in [t^* - 1]$, it computes $B_i^* = g^{f(idx_i^*)} \hat{g}_2^{y_{i,1}}$. For $i \in [t^*, n^*]$, it computes $B_i^* = g^{f(idx_i^*)} D^{\rho_{i,1}} \hat{g}_2^{\rho_{i,2}}$
4. It simulates a proof ψ^* of NIZK-WF.

Then $\mathcal{R}$ flips a bit $b \in \{0, 1\}$ to produce a session key that is a random element $K_0^* \in \mathbb{G}$ (if $\hat{b} = 0$) or $K_1^* = \hat{g}_2^{\hat{r}}$ (otherwise). $\mathcal{R}$ finally sends both CH^* and K_b^* to $\mathcal{A}$.

Query 2: Same as Query 1 with the restriction that CH^* cannot be asked as a share decapsulation query.

Guess: At the end, $\mathcal{A}$ outputs a guess $b' \in \{0,1\}$.

If $D = g^{\alpha\beta}$, $\mathcal{R}$ simulates $\mathbf{G}_1$. This is because for $i \in [t^*, n^*]$, we have $B_i^* = g^{f(idx_i^*)} D^{\rho_{i,1}} \hat{g}_2^{\rho_{i,2}} = g^{f(idx_i^*)} g^{\alpha\beta\rho_{i,1}+\beta\rho_{i,2}} = g^{f(idx_i^*)} Y_i^{\beta}$. Otherwise, each B_i^*, where $i \in [t^*, n^*]$, has a random distribution that is independent of each other. Thus, $\mathcal{R}$ simulates $\mathbf{G}_2$. Based on this, any distinguisher $\mathbf{G}_1$ and $\mathbf{G}_2$ can be converted into a DDH solver, and thereby, $|\Pr[\mathbf{S}_2] - \Pr[\mathbf{S}_3]| \leq \mathbf{Adv}_{DDH}(\lambda)$. □

Lemma 4.3. *If the DDH assumption holds, then no PPT algorithm can distinguish between $\boldsymbol{G}_4$ and $\boldsymbol{G}_5$.*

Proof. Let $\mathcal{R}$ be a reduction acting as a challenger to a SE-IND-CCA adversary $\mathcal{A}$. $\mathcal{R}$ interacts with $\mathcal{A}$ as follows:

Setup: $\mathcal{R}$ first generates $PP = (\mathbb{G}, p, g, h, H_0, H_1, H_2, H_3, H_4)$, where $h = g^{\hat{r}}$ for $\hat{r} \xleftarrow{\$} \mathbb{Z}_p$, the hash functions are modeled as random oracles, and the other components are as in the real scheme. Given PP, $\mathcal{A}$ submits t^* and $PL_C^* = \{PK_1^*, \ldots, PK_{t^*-1}^*\}$. For each $PK_j^* = (X_j, Y_j, \pi_j) \in PL_C^*$, $\mathcal{R}$ could obtain the corresponding witnesses (x_j, y_j) by extracting the proofs. After that, for $i \in [t^* - 1]$, $\mathcal{R}$ sets $H_0(PK_i^*) = idx_i^*$, where idx_i^* is chosen at random, by programming the random oracle. $\mathcal{R}$ then computes the Lagrange basis $L_i^*(\gamma) = \frac{\gamma}{idx_i^*} \prod_{j \in [t^*-1] \setminus \{i\}} \frac{\gamma - idx_j^*}{idx_i^* - idx_j^*}$ and also computes $L_0^*(\gamma) = \prod_{j \in [t^*-1]} \frac{\gamma - idx_j^*}{-idx_j^*}$. Let $\mathcal{U}_H$ be an empty set. $\mathcal{R}$ proceeds as follows:

1. It chooses random $y_i, \rho_i \in \mathbb{Z}_p$ and stores them to deal with the challenge phase.
2. It computes $X_i = \hat{g}_1^{L_0^*(idx_i)} g^{\rho_i}$ and $Y_i = g^{y_i}$.
3. It simulates a proof π_i of NIZK-PoK.
4. It adds $PK_i = (X_i, Y_i, \pi_i)$ to $\mathcal{U}_H$

In this way, although $\mathcal{R}$ does not know $x_i = L_0^*(idx_i^*)\alpha + \rho_i$, this is well simulated as ρ_i is uniformly random from $\mathcal{A}$'s point of view. Once the generation of $\mathcal{U}_H$ is completed, $\mathcal{R}$ sends it to $\mathcal{A}$.

Query 1: When $\mathcal{A}$ requests for some new input to the hash oracles, $\mathcal{R}$ returns a random value, except when simulating the NIZK proofs. To respond to a share decapsulation query for (PL, CH, PK_i), where $\mathbf{CHVerify}(PP, PL, CH) = 1$ and $PK_i \in \mathcal{U}_H$, $\mathcal{R}$ uses $y_i \in SK_i$ to return $\mu_i = \mathbf{ShareDecaps}(PP, PL, CH, SK_i)$.

Challenge: $\mathcal{A}$ requests a challenge for $PL_H^* \in \mathcal{U}_H$ with t^*. Consider $PL_H^* = \{PK_{t^*}^*, \ldots, PK_{n^*}^*\}$ and $H_0(PK_i^*) = idx_i^*$. $\mathcal{R}$ randomly picks $\eta, s_1, \ldots, s_{t^*-1} \in \mathbb{Z}_p$ and defines a degree $t^* - 1$ polynomial $f(\gamma) = L_0^*(\gamma)(\eta - \alpha\beta) + \sum_{i=1}^{t^*-1} L_i^*(\gamma) s_i$. This polynomial can be constructed by interpolating t^* points such that $f(0) = \eta - \alpha\beta$ and $f(idx_i^*) = s_i$ for all $i \in [1, t^* - 1]$. Now, $CH^* = (C_1^*, C_2^*, (A_1^*, B_1^*), \ldots, (A_{n^*}^*, B_{n^*}^*), t^*, \psi^*)$ is generated as follows:

1. It generates $C_1^* = \hat{g}_2^{\hat{r}} g^{\eta} D^{-1}$ and $C_2^* = \hat{g}_2$.
2. For $i \in [1, t^* - 1]$, it computes $A_i^* = g^{s_i} \hat{g}_2^{x_i}$. For $i \in [t^*, n^*]$, it computes $A_i^* = g^{L_0^*(idx_i^*)\eta + \sum_{j=1}^{t^*-1} L_j^*(idx_i^*) s_j} \hat{g}_2^{\rho_i}$.
3. For $i \in [1, t^* - 1]$, it computes $B_i^* = g^{s_i} \hat{g}_2^{y_i}$. For $i \in [t^*, n^*]$, it sets $B_i^* = R_i$ where $R_i \xleftarrow{\$} \mathbb{G}$.
4. It simulates a proof ψ^* of NIZK-WF.

Recall that $\mathcal{R}$ knows (x_i, y_i), where $i \in [1, t^*-1]$, extracted during the setup phase. Finally, $\mathcal{R}$ sends CH^* and K_b^*, where K_b^* is determined as $K_0^* = R \xleftarrow{\$} \mathbb{G}$ or $K_1^* = \hat{g}_2^{\hat{r}}$, depending on a random bit $b \in \{0,1\}$.

Query 2: Same as Query 1, with the restriction that CH^* cannot be asked as a share decapsulation query.

Guess: At the end, $\mathcal{A}$ outputs a guess $b' \in \{0,1\}$.

Consider $C_2^* = g^r$ (i.e., $\beta = r$). The simulation of (A_i^*, B_i^*) in CH^* is correct due to the following reason:

- For $i \in [1, t^*-1]$, $A_i^* = g^{s_i} \hat{g}_2^{x_i} = g^{f(idx_i^*)} X_i^r$ and $B_i^* = g^{s_i} \hat{g}_2^{y_i} = g^{f(idx_i^*)} Y_i^r$.
- For $i \in [t^*, n^*]$, $A_i^* = g^{L_0^*(idx_i^*)\eta + \sum_{j=1}^{t^*-1} L_j^*(idx_i^*) s_j} \hat{g}_2^{\rho_i}$
  $= g^{L_0^*(idx_i^*)(\eta - \alpha\beta) + \sum_{j=1}^{t^*-1} L_j^*(idx_i^*) s_j} \left(g^{L_0^*(idx_i^*)\alpha} g^{\rho_i}\right)^{\beta}$
  $= g^{L_0^*(idx_i^*) f(0) + \sum_{j=1}^{t^*-1} L_j^*(idx_i^*) f(idx_j^*)} (\hat{g}_1^{L_0^*(idx_i^*)} g^{\rho_i})^{\beta}$
  $= g^{f(idx_i^*)} X_i^r$.

If $D = g^{\alpha\beta}$, $\mathcal{R}$ simulates $\mathbf{G}_4$, since $C_1^* = \hat{g}_2^{\hat{r}} g^{\eta}(D^{-1}) = (g^{\beta})^{\hat{r}} g^{\eta - \alpha\beta} = h^r g^{f(0)}$ where $f(0) = \eta - \alpha\beta$. Otherwise, C_1^* is uniformly random, resulting in $\mathcal{R}$ which simulates $\mathbf{G}_5$. Therefore, any distinguisher $\mathbf{G}_4$ and $\mathbf{G}_5$ can be converted into a DDH solver. Thus $|\Pr[\mathbf{S}_4] - \Pr[\mathbf{S}_5]| \leq \mathbf{Adv}_{DDH}(\lambda)$. □

Lemma 4.4. *If the DDH assumption holds, then no PPT algorithm can distinguish between $\mathbf{G}_5$ and $\mathbf{G}_6$.*

Proof. Given a DDH tuple $(g, \hat{g}_1 = g^{\alpha}, \hat{g}_2 = g^{\beta}, D)$, a reduction $\mathcal{R}$ interacts with a selective IND-CCA adversary $\mathcal{A}$ as follows:

Setup: $\mathcal{R}$ sets $h = \hat{g}_1$ and generates public parameters $PP = (\mathbb{G}, p, g, h, H_0, H_1, H_2, H_3, H_4)$, where hash functions are modeled as random oracles. After $\mathcal{R}$ sends PP to $\mathcal{A}$, the adversary responds with t^* and $PL_C^* = \{PK_1^*, \ldots, PK_{t^*-1}^*\}$. For each PK_j^*, $\mathcal{R}$ extracts the witness (x_j, y_j) from π_j. $\mathcal{R}$ generates $\mathcal{U}_H$, initially set as empty, by following the process:

1. It randomly chooses $x_i, y_i \in \mathbb{Z}_p$ to compute $X_i = g^{x_i}$ and $Y_i = g^{y_i}$.
2. It simulates a proof π_i of NIZK-PoK.
3. Is adds $PK_i = (X_i, Y_i, \pi_i)$ to $\mathcal{U}_H$.

Since the tuple (X_i, Y_i) is computed as in the real scheme, this simulation is correct.

Query 1: The hash oracles are managed normally, by returning a random value for a new input except when simulating NIZK proofs. When $\mathcal{A}$ requests a share decapsulation query for (PL, CH, PK_i), where $\mathbf{CHVerify}(PP, PL, CH) = 1$ and $PK_i \in \mathcal{U}_H$, $\mathcal{R}$ returns $\mu_i = \mathbf{ShareDecaps}(PP, PL, CH, SK_i)$, especially using $y_i \in SK_i$.

Challenge: When $\mathcal{A}$ submits PL_H^* and t^*, $\mathcal{R}$ sets $PL^* = PL_C^* \cup PL_H^* = \{PK_1^*, \ldots, PK_{n^*}^*\}$ and obtains $H_0(PK_i^*) = idx_i^*$ for all PK_i^*. After that, $\mathcal{R}$ generates a (t^*-1)-degree polynomial $f(\gamma) = \sum_{j=0}^{t^*-1} s_j \gamma^j$, where $s_0, \ldots, s_{t^*-1} \in \mathbb{Z}_p$ are chosen randomly. $CH^* = (C_1^*, C_2^*, (A_1^*, B_1^*), \ldots, (A_{n^*}^*, B_{n^*}^*), t^*, \psi^*)$ is generated as follows:

1. It sets $C_1^* = R_1$, where $R_1 \xleftarrow{\$} \mathbb{G}$, and $C_2^* = \hat{g}_2$.
2. For $i \in [n^*]$, it computes $A_i^* = g^{f(idx_i^*)} \hat{g}_2^{x_i}$.

3. For $i \in [1, t^*-1]$, it computes $B_i^* = g^{f(idx_i^*)}\hat{g}_2^{y_i}$. For $i \in [t^*, n^*]$, it sets $B_i^* = R_i$, where $R_i \xleftarrow{\$} \mathbb{G}$.

4. Finally, it simulates a proof ψ^* of NIZK-WF.

Depending on a randomly chosen $b \in \{0,1\}$, $\mathcal{R}$ determines either $K_0^* \in \mathbb{G}$ or $K_1^* = D$. $\mathcal{R}$ then sends both CH^* and K_b^* to $\mathcal{A}$.

Query 2: Same as Query 1 with the restriction that CH^* cannot be asked as a share decapsulation query.

Guess: At the end, $\mathcal{A}$ outputs a guess b'.

If $D = g^{\alpha\beta}$, $\mathcal{R}$ simulates $\mathbf{G}_5$. This is because $K_1^* = (g^\alpha)^\beta = h^r$ by letting $\beta = r$. Otherwise $\mathcal{R}$ simulates $\mathbf{G}_6$. Based on this, any distinguisher $\mathbf{G}_5$ and $\mathbf{G}_6$ can be converted into a DDH solver, and thus $|\Pr[\mathbf{S}_5] - \Pr[\mathbf{S}_6]| \leq \mathbf{Adv}_{DDH}(\lambda)$. □

Theorem 4.5. *The proposed TKEM is DC secure if the underlying NIZK-EDL and NIZK-WF systems are both sound.*

Proof. Suppose that there exists a PPT adversary $\mathcal{A}$ that can break our TKEM in the DC security game. We can construct a reduction $\mathcal{R}$ (acting as a challenger) that interacts with $\mathcal{A}$ as follows:

Setup : $\mathcal{R}$ first gives $PP \leftarrow \mathbf{Setup}(1^\lambda)$ to $\mathcal{A}$. It also give $\mathcal{U}_H$ that includes a poly-bounded number of $(PK_i = (X_i, Y_i), SK_i = (x_i, y_i)) \leftarrow \mathbf{KeyGen}(PP)$. In response, $\mathcal{R}$ is given $\mathcal{U}_C$ from $\mathcal{A}$.

Query : $\mathcal{A}$ may request share decapsulation queries for (CH, PL, PK_i), where $PL \subseteq \mathcal{U}_H \cup \mathcal{U}_C$ and $PK_i \in \mathcal{U}_H$. Then, $\mathcal{R}$ sends either $\perp$ or μ_i, which can be obtained by running $\mathbf{ShareDecaps}(PP, PL, CH, SK_i)$.

Forge : $\mathcal{A}$ outputs a ciphertext header CH^* for a public key list $PL^* = \{PK_1^*, \ldots, PK_{n^*}^*\} \subset \mathcal{U}_H \cup \mathcal{U}_C$, a threshold t^*, and two sets of shares $\{\mu_i\}_{i\in S}$ and $\{\mu'_j\}_{j\in S'}$, where $t^* \leq |S| = |S'| \leq n^*$ and $S, S' \subset [n^*]$.

Assume that $\mathcal{A}$ wins with $(CH^*, PL^*, t^*, \{\mu_i\}_{i\in S}, \{\mu'_j\}_{j\in S'})$ that meets the three winning condition in Definition 3.3. Let $CH^* = (C_1^*, C_2^*, (A_1^*, B_1^*), \ldots, (A_{n^*}^*, B_{n^*}^*), \psi^*)$, $\mu_i = (D_i, \sigma_i)$, and $\mu'_j = (D'_j, \sigma'_j)$. Due to the condition (3), $\mathbf{CHVerify}(PP, PL^*, CH^*) = 1$. This means CH is well-formed as ψ^* of NIZK-WF is sound. Therefore, we can write $C_1^* = h^r g^{f(0)}$, $C_2^* = g^r$, $A_i^* = g^{f(idx_i^*)}(g^{x_i})^r$, and $B_i^* = g^{f(idx_i^*)}(g^{y_i})^r$ for the same $r \in \mathbb{G}$ and the same polynomial $f(\gamma)$. The condition (2) ensures that σ_i and σ'_j of NIZK-EDL are valid. Since these proofs are all sound, we have the strcutrues $A_i^*/D_i = (C_2^*)^{x_i}$ and $A_j^*/D'_j = (C_2^*)^{x_j}$, which can be rewritten as $D_i = A_i^*/(C_2^*)^{-x_i}$ and $D'_j = A_j^*/(C_2^*)^{-x_j}$, respectively. We now show that the condition (1) never holds, which contradicts the hypothesis: Consider $K = \mathbf{Combine}(PP, PL^*, CH^*, \{\mu_i\}_{i\in S})$ and $K' = \mathbf{Combine}(PP, PL^*, CH^*, \{\mu'_j\}_{j\in S'})$. Specifically, K is computed as follows:

$$\begin{aligned} K &= C_1^* \prod_{i\in S} (D_i)^{-L_i(0)} = (h^r g^{f(0)}) \prod_{i\in S} (g^{f(idx_i^*)} X_i^r / (g^r)^{-x_i})^{-L_i(0)} \\ &= (h^r g^{f(0)}) \prod_{i\in S} (g^{f(idx_i^*)})^{-L_i(0)} = (h^r g^{f(0)}) g^{-f(0)} = h^r. \end{aligned}$$

Similarly, we can have $K' = C_1^* \prod_{j\in S'} (D'_j)^{-L_j(0)} = h^r$, leading to $K = K'$.

Consequently, $\mathcal{A}$ must compromise the soundness of NIZK-WF or NIZK-EDL to win the DC security game, resulting in the advantage of $\mathcal{A}$ as follows.

$$\mathbf{Adv}_{TKEM}^{DC}(\lambda) \leq \mathbf{Adv}_{NIZK-EDL}^{Sound}(\lambda) + \mathbf{Adv}_{NIZK-WF}^{Sound}(\lambda).$$

This completes the proof.

□

Table 3: Time evaluation of our TKEM

n	**Setup**(ms)	**Keygen**(s)	**Encaps**(s)	**CHVerify**(s)	**ShareDecap**(s)	**ShareVerify**(s)	**Combine**(s)
100	0.162	0.043	14.403	7.925	1.925	0.095	0.744
200	0.181	0.045	29.100	16.521	1.916	0.096	1.847
300	0.177	0.043	44.956	27.173	1.944	0.096	3.202
400	0.164	0.042	64.267	41.232	1.951	0.096	4.907
500	0.169	0.043	89.479	61.041	1.829	0.092	6.809

5 Performance Analysis

We analyze the performance of our TKEM. First, we describe the implementation environment, followed by the presentation of our experimental results. In particular, we compare our scheme with SES16a, the TPKE scheme of Sakai et al. [42]. To the best of our knowledge, it is the only existing dynamic threshold system that supports a transparent setup.

5.1 Environment

The performance evaluation was conducted on a machine running Windows 10 Pro with an Intel Core i7-8700 CPU @ 3.70 GHz and 16 GB DDR4 RAM. All implementations were written in Python using two cryptographic libraries: *Miracle-core* [1] and *Charm-Crypto* [3]. The Miracle-core library was used to implement our TKEM as a proof-of-concept. It provides optimized support for prime-order elliptic curves, such as secp256k1, without requiring pairing. However, it does not support pairing-friendly curves used in symmetric pairings (e.g., the SS1024 curve [40]), which is required to instantiate SES16a. For this reason, we used the the Charm-Crypto library for a fair comparison of the computational efficiency. To ensure consistency, we measured the costs of basic cryptographic operations on both the secp256k1 and SS1024 curves. Both curves provide at least 112-bit security [40].

5.2 Implementation

We present a proof-of-concept implementation of our TKEM. Tables 3 and 4 show the average execution times for each algorithm, obtained by running the corresponding algorithm multiple times with n receivers, where $n \in \{100, 200, 300, 400, 500\}$, under the threshold parameter $t = n/2$. In Table 3, the performance results indicate that the time complexity of all algorithms, except the **Setup** algorithm, increases approximately linearly with a small number of receivers. Notably, Table 3 and Table 4, the **Encaps** and **CHVerify** algorithms together account for more than 70% of the total execution time. This is primarily due to the generation of a parity check matrix $\mathbf{H}$. These generation is involved in the NIZK-WF algorithms, which are crucial to the contributions of our TKEM. Although our TKEM construction does not rely on pairing operations, the computations involved in generating the matrix $\mathbf{H}$, such as group exponentiations and scalar multiplications, remain the primary performance bottlenecks. Fortunately, we apply an optimization technique to mitigate this issue, which is discussed in Section 6.

Table 4: Time evaluation of the employed NIZK-WF algorithms

n	**WF.Prove**(s)	**WF.Verify**(s)
100	7.058	7.925
200	14.559	16.521
300	23.690	27.173
400	36.215	41.232
500	54.474	61.041

Table 5: K cycle Counts per Operation for secp256k1 and SS1024

Operation	secp256k1	SS1024
$E_{\mathbb{Z}_P}$	76	1,527
$M_{\mathbb{Z}_p}$	4	3
$E_{\mathbb{G}}$	1,230	45,222
$M_{\mathbb{G}}$	3	38
$E_{\mathbb{G}_T}$	–	5,310
$M_{\mathbb{G}_T}$	–	9
P	–	66,282

E_X and M_X indicate an exponentiation and a multiplication in X where $X \in \{\mathbb{Z}_p, \mathbb{G}, \mathbb{G}_T\}$. P indicates a pairing operation.

5.3 Comparison with SES16a

Our TKEM avoids pairing operations entirely by using a standard cyclic group, whereas SES16a is constructed over a symmetric pairing-friendly curve. For a fair comparison, we analyze each construction by counting the number of required core operations, such as group operations, exponentiations, and scalar operations, as shown in Table 6. We then measure the average cycle cost of each operation to assess computational efficiency. Note that scalar operations are typically considered negligible relative to group operations (and also pairing operations) and are often excluded from performance evaluations. However, our implementation results indicate that they can cause noticeable performance bottlenecks. Therefore, we include them in our analysis.

Importantly, Table 5 shows that identical operations can incur significantly different costs depending on the underlying curve. This is because secp256k1 is defined over a group of 256-bit prime order, whereas SS1024 operates over a group of 1024-bit order. This difference has a substantial impact on the cost per operation. As shown in Table 6, our TKEM consistently demonstrates significantly lower cycle costs across most algorithms. In particular, the **Encaps** algorithm in SES16a suffers from heavy pairing-related overhead; for example, it requires over 24 million K cycles, which is more than 15 times the cost of ours (approximately 1.6 million K cycles). Although our **Combine** algorithm is slower than that of SES16a, this is executed only once after all valid shares have been collected. Therefore, the increased cost may acceptable in practice. We

Table 6: Computation Cost Comparison between our TKEM and SES16a

	KeyGen								**Encaps**							
Scheme	$E_{\mathbb{Z}_p}$	$M_{\mathbb{Z}_p}$	E_G	M_G	E_{G_T}	M_{G_T}	P	K cycle	$E_{\mathbb{Z}_p}$	$M_{\mathbb{Z}_p}$	E_G	M_G	E_{G_T}	M_{G_T}	P	K cycle
SES16a	–	–	1	–	–	–	–	45,223	$t(t+1)$	$t(t+1)+3$	$3n+4$	$n+1$	0	n	n	23,514,675
Ours	2	0	4 †	–	–	–	–	4,926	$n(t-1)$	$n^2+n(t-2)+1$ †	$9n+18$	$7n+6$	–	–	–	1,477,713
	CHVerify								**ShareDecaps**							
Scheme	$E_{\mathbb{Z}_p}$	$M_{\mathbb{Z}_p}$	E_G	M_G	E_{G_T}	M_{G_T}	P	K cycle	$E_{\mathbb{Z}_p}$	$M_{\mathbb{Z}_p}$	E_G	M_G	E_{G_T}	M_{G_T}	P	K cycle
SES16a	Not Supported							–	–	–	7	4	1	5	–	648,131
Ours	n^2+1 †	$8n+10$	$7n+1$	–	–	–	–	1,021,130	2	6	1	–	–	–	–	7,390
	ShareVerify								**Combine**							
Scheme	$E_{\mathbb{Z}_p}$	$M_{\mathbb{Z}_p}$	E_G	M_G	E_{G_T}	M_{G_T}	P	K cycle	$E_{\mathbb{Z}_p}$	$M_{\mathbb{Z}_p}$	E_G	M_G	E_{G_T}	M_{G_T}	P	K cycle
SES16a	–	–	4	3	1	1	5	517,736	–	$2t(2t-1)$	–	–	–	–	–	32,017
Ours	1	8	4	–	–	–	–	9,860	–	$2t(t-1)$	t	t	–	–	–	72,450

† indicates the operations optimized as described in Section 6.

highlight that these results clearly demonstrate the computational advantage of our pairing-free design.

6 Discussion

Efficiency. Recall that our TKEM incurs linear overhead in both the ciphertext header size and the number of NIZK proofs: To address this, our implementation incorporates several optimizations to mitigate performance bottlenecks. Notably, computing the Lagrange coefficient matrix $\mathbf{H} = \left(\frac{1}{\prod_{\ell\in[0,n],\ell\neq j}\gamma_j-\gamma_\ell}\gamma_j^i\right)_{i,j} \in \mathbb{Z}_p^{(n-t+1)\times(n+1)}$ requires $O(n^3)$ multiplications in a naïve approach. However, since the denominator depends only on the column index j, it can be precomputed and reused across rows. This approach reduces the total cost to $O(n^2)$. In addition, when ciphertexts are generated repeatedly for a fixed set of users, $\mathbf{H}$ can be completely precomputed for further efficiency. Our security proof relies on a straight-line extractable NIZK, while the implementation uses a Fiat-Shamir transformation of a Sigma protocol. Although this prevents a rigorous security analysis due to the repeated use of rewinding, it remains widely accepted for its simplicity and efficiency. Moreover, Fiat-Shamir NIZK proof systems have demonstrated strong heuristic security in real-world applications, especially when modeled in the ROM. Despite these optimizations, unfortunately, the $O(n)$ ciphertext header size remains a scalability issue in large-scale settings.

Adaptive Security. The proposed TKEM guarantees security only in the selective IND-CCA model. This limitation arises because the challenger must embed a DDH instance into the challenger ciphertext header. To do so, it requires to know the target user set at the beginning of the security game. Specifically, the challenger constructs a polynomial for Lagrange interpolation over the selected indices. This polynomial is used to consistently generate the ciphertext header and related public keys. In principle, the TKEM could support an adaptive security by requiring the challenger to guess the target set beforehand. Unfortunately, the probability of guessing correctly is only $1/\binom{n}{t-1}$, which becomes negligible as n increases. Adaptive security is a stronger and more realistic notion, as it capture dynamic adversarial behavior in practical settings. While achieving adaptive security in the non-dynamic threshold setting has typically relied on pairing-based constructions [37] or stronger assumptions such as reliable erasure [12], recent works have successfully realized it in threshold signature schemes [4, 5, 15, 16]. Extending these advances to the TKEM remains an important and promising direction. Achieving adaptive IND-CCA security in a dynamic and pairing-free TKEM would not only strengthen the theoretical guarantees but also improve its applicability

in real-world deployment.

Side Channel Attacks. Indeed, our TKEM currently does not consider implementation-level threats, such as side-channel leakage during group exponentiation. In practice, such operations are known to be vulnerable to timing and power analysis attacks [34, 35]. These attacks, however, can be mitigated at the implementation level through well-established countermeasures, such as algorithmic blinding and constant-time computation [31, 32]. Beyond these engineering techniques, an important direction for future work is to extend the formal security model to capture a PPT adversary with side-channel capabilities explicitly; e.g., [24].

7 Conclusion

In this paper, we propose a dynamic TKEM with a transparent setup that avoids the use of pairings. One of our main result is a novel NIZK-WF proof system with compact proof size. Our design supports fully dynamic thresholds, which enhances flexibility in decentralized settings. We also prove the selective IND-CCA security and decapsulation consistency of our TKEM under the DDH assumption, along with the security of the underlying NIZK proof systems derived from Sigma protocols in the random oracle model. While our work provides a solid theoretical foundation, future directions include improving scalability, developing a post-quantum variants, and achieving adaptive security.

References

[1] Miracl core library. `https://github.com/miracl/core`.

[2] Shweta Agrawal, Damien Stehlé, and Anshu Yadav. Round-optimal lattice-based threshold signatures, revisited. In Mikolaj Bojanczyk, Emanuela Merelli, and David P. Woodruff, editors, *49th International Colloquium on Automata, Languages, and Programming, ICALP 2022*, volume 229 of *LIPIcs*, pages 8:1–8:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.

[3] Joseph A. Akinyele, Christina Garman, Ian Miers, Matthew W. Pagano, Michael Rushanan, Matthew Green, and Aviel D. Rubin. Charm: a framework for rapidly prototyping cryptosystems. *J. Cryptogr. Eng.*, 3(2):111–128, 2013.

[4] Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Adaptively secure three-round threshold schnorr signatures from DDH. Cryptology ePrint Archive, Paper 2025/1009, 2025.

[5] Renas Bacho, Sourav Das, Julian Loss, and Ling Ren. Glacius: Threshold schnorr signatures from DDH with full adaptive security. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology - EUROCRYPT 2025 - 44th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Madrid, Spain, May 4-8, 2025, Proceedings, Part II*, volume 15602 of *Lecture Notes in Computer Science*, pages 304–334. Springer, 2025.

[6] Renas Bacho, Julian Loss, Stefano Tessaro, Benedikt Wagner, and Chenzhi Zhu. Twinkle: Threshold signatures from DDH with full adaptive security. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology - EUROCRYPT 2024, Part I*, volume 14651 of *Lecture Notes in Computer Science*, pages 429–459. Springer, 2024.

[7] Mihir Bellare, Elizabeth C. Crites, Chelsea Komlo, Mary Maller, Stefano Tessaro, and Chenzhi Zhu. Better than advertised security for non-interactive threshold signatures. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology - CRYPTO 2022*, volume 13510 of *Lecture Notes in Computer Science*, pages 517–550. Springer, 2022.

[8] Mihir Bellare, Georg Fuchsbauer, and Alessandra Scafuro. Nizks with an untrusted CRS: security in the face of parameter subversion. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part II*, volume 10032 of *Lecture Notes in Computer Science*, pages 777–804, 2016.

[9] Mihir Bellare, Stefano Tessaro, and Chenzhi Zhu. Stronger security for non-interactive threshold signatures: BLS and FROST. Cryptology ePrint Archive, Paper 2022/833, 2022. `https://eprint.iacr.org/2022/833`.

[10] Dan Boneh, Ben Lynn, and Hovav Shacham. Short signatures from the Weil pairing. In Colin Boyd, editor, *Advances in Cryptology - ASIACRYPT 2001*, volume 2248 of *Lecture Notes in Computer Science*, pages 514–532. Springer, 2001.

[11] Jan Camenisch and Markus Stadler. Efficient group signature schemes for large groups (extended abstract). In Burton S. Kaliski Jr., editor, *Advances in Cryptology - CRYPTO '97*, volume 1294 of *Lecture Notes in Computer Science*, pages 410–424. Springer, 1997.

[12] Ran Canetti, Rosario Gennaro, Stanislaw Jarecki, Hugo Krawczyk, and Tal Rabin. Adaptive security for threshold cryptosystems. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 98–115. Springer, 1999.

[13] Rutchathon Chairattana-Apirom, Stefano Tessaro, and Chenzhi Zhu. Partially non-interactive two-round lattice-based threshold signatures. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology - ASIACRYPT 2024 - 30th International Conference on the Theory and Application of Cryptology and Information Security, Kolkata, India, December 9-13, 2024, Proceedings, Part IV*, volume 15487 of *Lecture Notes in Computer Science*, pages 268–302. Springer, 2024.

[14] Arka Rai Choudhuri, Sanjam Garg, Julien Piet, and Guru-Vamsi Policharla. Mempool privacy via batched threshold encryption: Attacks and defenses. In Davide Balzarotti and Wenyuan Xu, editors, *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*. USENIX Association, 2024.

[15] Elizabeth Crites, Jonathan Katz, Chelsea Komlo, Stefano Tessaro, and Chenzhi Zhu. On the adaptive security of FROST. Cryptology ePrint Archive, Paper 2025/1061, 2025.

[16] Elizabeth Crites and Alistair Stewart. A plausible attack on the adaptive security of threshold schnorr signatures. Cryptology ePrint Archive, Paper 2025/1001, 2025.

[17] Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. Fully adaptive Schnorr threshold signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023, Part I*, volume 14081 of *Lecture Notes in Computer Science*, pages 678–709. Springer, 2023.

[18] Sourav Das, Philippe Camacho, Zhuolun Xiang, Javier Nieto, Benedikt Bünz, and Ling Ren. Threshold signatures from inner product argument: Succinct, weighted, and multi-threshold. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023*, pages 356–370. ACM, 2023.

[19] Cécile Delerablée and David Pointcheval. Dynamic threshold public-key encryption. In David A. Wagner, editor, *Advances in Cryptology - CRYPTO 2008*, volume 5157 of *Lecture Notes in Computer Science*, pages 317–334. Springer, 2008.

[20] Yvo Desmedt. Threshold cryptography. *Eur. Trans. Telecommun.*, 5(4):449–458, 1994.

[21] Nico Döttling, Lucjan Hanzlik, Bernardo Magri, and Stella Wohnig. Mcfly: Verifiable encryption to the future made practical. In Foteini Baldimtsi and Christian Cachin, editors, *Financial Cryptography and Data Security - FC 2023*, volume 13950 of *Lecture Notes in Computer Science*, pages 252–269. Springer, 2023.

[22] Thomas Espitau, Shuichi Katsumata, and Kaoru Takemure. Two-round threshold signature from algebraic one-more learning with errors. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VII*, volume 14926 of *Lecture Notes in Computer Science*, pages 387–424. Springer, 2024.

[23] Thomas Espitau, Guilhem Niot, and Thomas Prest. Flood and submerse: Distributed key generation and robust threshold signature from lattices. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VII*, volume 14926 of *Lecture Notes in Computer Science*, pages 425–458. Springer, 2024.

[24] Sebastian Faust, Markulf Kohlweiss, Giorgia Azzurra Marson, and Daniele Venturi. On the non-malleability of the fiat-shamir transform. In Steven D. Galbraith and Mridul Nandi, editors, *Progress in Cryptology - INDOCRYPT 2012, 13th International Conference on Cryptology in India, Kolkata, India, December 9-12, 2012. Proceedings*, volume 7668 of *Lecture Notes in Computer Science*, pages 60–79. Springer, 2012.

[25] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology - CRYPTO '86, Santa Barbara, California, USA, 1986, Proceedings*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194. Springer, 1986.

[26] Marc Fischlin. Communication-efficient non-interactive proofs of knowledge with online extractors. In Victor Shoup, editor, *Advances in Cryptology - CRYPTO 2005: 25th Annual International Cryptology Conference, Santa Barbara, California, USA, August 14-18, 2005, Proceedings*, volume 3621 of *Lecture Notes in Computer Science*, pages 152–168. Springer, 2005.

[27] Sanjam Garg, Abhishek Jain, Pratyay Mukherjee, Rohit Sinha, Mingyuan Wang, and Yinuo Zhang. hints: Threshold signatures with silent setup. In *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19-23, 2024*, pages 3034–3052. IEEE, 2024.

[28] Sanjam Garg, Dimitris Kolonelos, Guru-Vamsi Policharla, and Mingyuan Wang. Threshold encryption with silent setup. In Leonid Reyzin and Douglas Stebila, editors, *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part VII*, volume 14926 of *Lecture Notes in Computer Science*, pages 352–386. Springer, 2024.

[29] Jens Groth and Amit Sahai. Efficient non-interactive proof systems for bilinear groups. In Nigel P. Smart, editor, *Advances in Cryptology - EUROCRYPT 2008*, volume 4965 of *Lecture Notes in Computer Science*, pages 415–432. Springer, 2008.

[30] Kamil Doruk Gür, Jonathan Katz, and Tjerand Silde. Two-round threshold lattice-based signatures from threshold homomorphic encryption. In Markku-Juhani O. Saarinen and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - PQCrypto 2024, Part II*, volume 14772 of *Lecture Notes in Computer Science*, pages 266–300. Springer, 2023.

[31] Marc Joye and Christophe Tymen. Protections against differential analysis for elliptic curve cryptography. In Çetin Kaya Koç, David Naccache, and Christof Paar, editors, *Cryptographic Hardware and Embedded Systems - CHES 2001, Third International Workshop, Paris, France, May 14-16, 2001, Proceedings*, volume 2162 of *Lecture Notes in Computer Science*, pages 377–390. Springer, 2001.

[32] Marc Joye and Sung-Ming Yen. The montgomery powering ladder. In Burton S. Kaliski Jr., Çetin Kaya Koç, and Christof Paar, editors, *Cryptographic Hardware and Embedded Systems - CHES 2002, 4th International Workshop, Redwood Shores, CA, USA, August 13-15, 2002, Revised Papers*, volume 2523 of *Lecture Notes in Computer Science*, pages 291–302. Springer, 2002.

[33] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *Advances in Cryptology - ASIACRYPT 2010 - 16th International Conference on the Theory and Application of Cryptology and Information Security, Singapore, December 5-9, 2010. Proceedings*, volume 6477 of *Lecture Notes in Computer Science*, pages 177–194. Springer, 2010.

[34] Paul C. Kocher. Timing attacks on implementations of diffie-hellman, rsa, dss, and other systems. In Neal Koblitz, editor, *Advances in Cryptology - CRYPTO '96, 16th Annual International Cryptology Conference, Santa Barbara, California, USA, August 18-22, 1996, Proceedings*, volume 1109 of *Lecture Notes in Computer Science*, pages 104–113. Springer, 1996.

[35] Paul C. Kocher, Joshua Jaffe, and Benjamin Jun. Differential power analysis. In Michael J. Wiener, editor, *Advances in Cryptology - CRYPTO '99, 19th Annual International Cryptology Conference, Santa Barbara, California, USA, August 15-19, 1999, Proceedings*, volume 1666 of *Lecture Notes in Computer Science*, pages 388–397. Springer, 1999.

[36] Chelsea Komlo and Ian Goldberg. FROST: flexible round-optimized Schnorr threshold signatures. In Orr Dunkelman, Michael J. Jacobson Jr., and Colin O'Flynn, editors, *Selected Areas in Cryptography - SAC 2020*, volume 12804 of *Lecture Notes in Computer Science*, pages 34–65. Springer, 2020.

[37] Benoît Libert and Moti Yung. Non-interactive cca-secure threshold cryptosystems with adaptive security: New framework and constructions. In Ronald Cramer, editor, *Theory of Cryptography - 9th Theory of Cryptography Conference, TCC 2012, Taormina, Sicily, Italy, March 19-21, 2012. Proceedings*, volume 7194 of *Lecture Notes in Computer Science*, pages 75–93. Springer, 2012.

[38] Moni Naor and Moti Yung. Public-key cryptosystems provably secure against chosen ciphertext attacks. In Harriet Ortiz, editor, *Proceedings of the 22nd Annual ACM Symposium on Theory of Computing*, pages 427–437. ACM, 1990.

[39] Rafaël Del Pino, Shuichi Katsumata, Mary Maller, Fabrice Mouhartem, Thomas Prest, and Markku-Juhani O. Saarinen. Threshold raccoon: Practical threshold signatures from standard lattice assumptions. In Marc Joye and Gregor Leander, editors, *Advances in Cryptology - EUROCRYPT 2024, Part II*, volume 14652 of *Lecture Notes in Computer Science*, pages 219–248. Springer, 2024.

[40] Yannis Rouselakis and Brent Waters. Efficient statically-secure large-universe multi-authority attribute-based encryption. In Rainer Böhme and Tatsuaki Okamoto, editors, *Financial Cryptography and Data Security - 19th International Conference, FC 2015, San Juan, Puerto Rico, January 26-30, 2015, Revised Selected Papers*, volume 8975 of *Lecture Notes in Computer Science*, pages 315–332. Springer, 2015.

[41] Tim Ruffing, Viktoria Ronge, Elliott Jin, Jonas Schneider-Bensch, and Dominique Schröder. ROAST: robust asynchronous Schnorr threshold signatures. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022*, pages 2551–2564. ACM, 2022.

[42] Yusuke Sakai, Keita Emura, Jacob C. N. Schuldt, Goichiro Hanaoka, and Kazuo Ohta. Constructions of dynamic and non-dynamic threshold public-key encryption schemes with decryption consistency. *Theor. Comput. Sci.*, 630:95–116, 2016.

[43] Jacob T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *J. ACM*, 27(4):701–717, 1980.

[44] Victor Shoup. Iso 18033-2: An emerging standard for public-key encryption. *http://shoup. net/iso/*, 2004.

[45] Victor Shoup and Rosario Gennaro. Securing threshold cryptosystems against chosen ciphertext attack. *J. Cryptol.*, 15(2):75–96, 2002.

[46] Stefano Tessaro and Chenzhi Zhu. Threshold and multi-signature schemes from linear hash functions. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology - EUROCRYPT 2023, Part V*, volume 14008 of *Lecture Notes in Computer Science*, pages 628–658. Springer, 2023.

[47] Richard Zippel. Probabilistic algorithms for sparse polynomials. In Edward W. Ng, editor, *Symbolic and Algebraic Computation, EUROSAM '79*, volume 72 of *Lecture Notes in Computer Science*, pages 216–226. Springer, 1979.